EXPERIMENT NO. 01

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Python code to get information about the file pertaining to the file mode and to get time values with components using local time and gm time.

Program:

```
import os
import time

# Function to get file information and times
def get_file_info(file_path):
    # Checking if the file exists
    if os.path.exists(file_path):
        # Get file stats
        file_stats = os.stat(file_path)

        # Getting file permissions (mode)
        file_mode = oct(file_stats.st_mode)[-3:]

        # Get times
        creation_time = file_stats.st_ctime  # Creation time
        modification_time = file_stats.st_mtime  # Modification time
        access_time = file_stats.st_atime  # Last access time

        # Convert times to local and GMT time formats
        local_creation_time = time.localtime(creation_time)
        gmt_creation_time = time.gmtime(creation_time)

        local_modification_time = time.localtime(modification_time)
        gmt_modification_time = time.gmtime(modification_time)

        local_access_time = time.localtime(access_time)
        gmt_access_time = time.gmtime(access_time)

        # Printing file information
        print(f"File: {file_path}")
        print(f"Permissions (mode): {file_mode}")
```

```
        print("\nTime Information:")
        print(f"Creation Time (Local): {time.strftime('%Y-%m-%d %H:%M:%S', local_creation_time)}")
        print(f"Creation Time (GMT): {time.strftime('%Y-%m-%d %H:%M:%S', gmt_creation_time)}")

        print(f"Modification Time (Local): {time.strftime('%Y-%m-%d %H:%M:%S', local_modification_time)}")
        print(f"Modification Time (GMT): {time.strftime('%Y-%m-%d %H:%M:%S', gmt_modification_time)}")

        print(f"Access Time (Local): {time.strftime('%Y-%m-%d %H:%M:%S', local_access_time)}")
        print(f"Access Time (GMT): {time.strftime('%Y-%m-%d %H:%M:%S', gmt_access_time)}")
    else:
        print(f"File '{file_path}' does not exist.")

# Example file path (replace with your file's path)
file_path = "sample.txt"  # Change this to a file that exists on your system
get_file_info(file_path)
```

OUTPUT:

```
File: sample.txt
Permissions (mode): 644

Time Information:
Creation Time (Local): 2025-09-23 07:42:20
Creation Time (GMT): 2025-09-23 07:42:20
Modification Time (Local): 2025-09-23 07:42:20
Modification Time (GMT): 2025-09-23 07:42:20
Access Time (Local): 2025-09-23 07:42:28
Access Time (GMT): 2025-09-23 07:42:28
```

EXPERIMENT NO. 02

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Perform Array operations using Numpy: Write a Python program to
i. Create N-dim arrays
ii. Use inbuilt function (zeros, ones, full, eye) to create array
iii. Use shape, reshape, transpose function
iv. Use sort function and Sorting along axes
v. Use arange, linspace, logspace function
vi. Array Indexing , slicing, Joining, Splitting, and Searching arrays
vii. Statistical Functions on arrays

Program:

```
import numpy as np

# i. Create N-dimensional arrays
arr_1d = np.array([1, 2, 3, 4, 5])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print("1D Array:\n", arr_1d)
print("2D Array:\n", arr_2d)
print("3D Array:\n", arr_3d)

# ii. Use inbuilt function (zeros, ones, full, eye) to create arrays
zeros_arr = np.zeros((3, 3))
ones_arr = np.ones((2, 2))
full_arr = np.full((3, 3), 7)  # Fill with 7
eye_arr = np.eye(4)  # Identity matrix

print("\nZeros Array:\n", zeros_arr)
print("Ones Array:\n", ones_arr)
print("Full Array (7s):\n", full_arr)
print("Identity Matrix (Eye):\n", eye_arr)

# iii. Use shape, reshape, transpose function
arr_reshaped = arr_2d.reshape((3, 2))
arr_transposed = arr_2d.T
```

```
print("\nReshaped Array:\n", arr_reshaped)
print("Transposed Array:\n", arr_transposed)

# iv. Use sort function and Sorting along axes
arr_to_sort = np.array([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
sorted_arr = np.sort(arr_to_sort)  # Sort in ascending order
sorted_2d_arr = np.sort(arr_2d, axis=0)  # Sort along column axis

print("\nSorted Array (1D):\n", sorted_arr)
print("Sorted Array (2D, axis=0):\n", sorted_2d_arr)

# v. Use arange, linspace, logspace function
arr_arange = np.arange(0, 10, 2)  # Creates array [0, 2, 4, 6, 8]
arr_linspace = np.linspace(0, 1, 5)  # Creates array with 5 equally
spaced values between 0 and 1
arr_logspace = np.logspace(1, 3, 3)  # Creates array with 3 values
spaced log10 between 10^1 and 10^3

print("\nArange Array:\n", arr_arange)
print("Linspace Array:\n", arr_linspace)
print("Logspace Array:\n", arr_logspace)

# vi. Array Indexing, Slicing, Joining, Splitting, and Searching arrays

# Indexing
index_value = arr_2d[1, 2]  # Access value at second row, third column
print("\nIndexed Value (arr_2d[1,2]):", index_value)

# Slicing
slice_arr = arr_2d[:, 1]  # Slice all rows, second column
print("Sliced Array (all rows, second column):", slice_arr)

# Joining
arr_joined = np.concatenate((arr_1d, arr_arange))
print("\nJoined Array:\n", arr_joined)

# Splitting
arr_split = np.split(arr_joined, 2)  # Split array into 2 equal parts
print("Split Array:", arr_split)

# Searching
search_result = np.where(arr_2d == 5)  # Search for elements equal to 5
```

```python
print("Search Result (positions of 5):", search_result)

# vii. Statistical Functions on arrays
arr_stat = np.array([1, 2, 3, 4, 5])
mean_value = np.mean(arr_stat)  # Mean
std_dev = np.std(arr_stat)  # Standard deviation
sum_value = np.sum(arr_stat)  # Sum
min_value = np.min(arr_stat)  # Min
max_value = np.max(arr_stat)  # Max

print("\nStatistical Functions:")
print(f"Mean: {mean_value}")
print(f"Standard Deviation: {std_dev}")
print(f"Sum: {sum_value}")
print(f"Min: {min_value}")
print(f"Max: {max_value}")
```

OUTPUT:

```
1D Array:
 [1 2 3 4 5]
2D Array:
 [[1 2 3]
 [4 5 6]]
3D Array:
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]

Zeros Array:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones Array:
 [[1. 1.]
 [1. 1.]]
Full Array (7s):
 [[7 7 7]
 [7 7 7]
 [7 7 7]]
Identity Matrix (Eye):
 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
Reshaped Array:
 [[1 2]
 [3 4]
 [5 6]]
Transposed Array:
 [[1 4]
 [2 5]
 [3 6]]

Sorted Array (1D):
 [1 1 2 3 3 4 5 5 5 6 9]
Sorted Array (2D, axis=0):
 [[1 2 3]
 [4 5 6]]

Arrange Array:
 [0 2 4 6 8]
Linspace Array:
 [0.   0.25 0.5  0.75 1.  ]
Logspace Array:
 [  10.  100. 1000.]

Indexed Value (arr_2d[1,2]): 6
Sliced Array (all rows, second column): [2 5]

Joined Array:
 [1 2 3 4 5 0 2 4 6 8]
Split Array: [array([1, 2, 3, 4, 5]), array([0, 2, 4, 6, 8])]
Search Result (positions of 5): (array([1]), array([1]))

Statistical Functions:
Mean: 3.0
Standard Deviation: 1.4142135623730951
Sum: 15
Min: 1
Max: 5
```

EXPERIMENT NO. 03

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Perform Data Manipulation operations using Pandas package on dataframes.

Program:

```
import pandas as pd
import numpy as np

# Sample data to demonstrate DataFrame operations
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [24, 27, 22, 32, 29],
    'City':   ['New   York',   'Los   Angeles',   'Chicago',   'Houston',
'Phoenix'],
    'Salary': [70000, 80000, 120000, 110000, 95000]
}

# Create a DataFrame
df = pd.DataFrame(data)

# i. Display the first few rows of the DataFrame
print("Original DataFrame:\n", df)

# ii. Select specific columns (e.g., Name and Age)
selected_columns = df[['Name', 'Age']]
print("\nSelected Columns (Name, Age):\n", selected_columns)

# iii. Filter rows based on a condition (e.g., Age > 25)
filtered_df = df[df['Age'] > 25]
print("\nFiltered Rows (Age > 25):\n", filtered_df)

# iv. Add a new column (e.g., Bonus based on salary)
df['Bonus'] = df['Salary'] * 0.1
print("\nDataFrame with Bonus column:\n", df)

# v. Drop a column (e.g., Bonus column)
df_dropped = df.drop('Bonus', axis=1)
print("\nDataFrame after Dropping Bonus column:\n", df_dropped)
```

```python
# vi. Grouping and Aggregating Data (e.g., average salary by city)
grouped_df = df.groupby('City')['Salary'].mean()
print("\nAverage Salary by City:\n", grouped_df)

# vii. Sorting DataFrame by a specific column (e.g., sort by Salary)
sorted_df = df.sort_values(by='Salary', ascending=False)
print("\nSorted DataFrame by Salary (Descending):\n", sorted_df)

# viii. Handling Missing Data (NaN)
# Adding missing data for demonstration
df_with_nan = df.copy()
df_with_nan.loc[2, 'Salary'] = np.nan  # Introducing NaN in Salary
print("\nDataFrame with Missing Value (NaN):\n", df_with_nan)

# Filling missing data with the mean value
df_filled = df_with_nan.fillna(df_with_nan['Salary'].mean())
print("\nDataFrame after Filling Missing Salary with Mean:\n",
df_filled)

# ix. Merging DataFrames (Creating a second DataFrame)
data2 = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Department': ['HR', 'IT', 'Finance']
}
df2 = pd.DataFrame(data2)

# Merging df and df2 on 'Name'
merged_df = pd.merge(df, df2, on='Name', how='inner')
print("\nMerged DataFrame:\n", merged_df)

# x. Concatenating DataFrames (Stacking vertically)
df_concatenated = pd.concat([df, df2], axis=0, ignore_index=True)
print("\nConcatenated DataFrame (Vertical Stack):\n", df_concatenated)

# xi. Reshaping DataFrame (Pivoting)
df_pivot = df.pivot_table(values='Salary', index='City', aggfunc='mean')
print("\nPivoted DataFrame (Average Salary by City):\n", df_pivot)
```

OUTPUT:

```
Original DataFrame:
       Name  Age         City  Salary
0    Alice   24     New York   70000
1      Bob   27  Los Angeles   80000
2  Charlie   22      Chicago  120000
3    David   32      Houston  110000
4      Eve   29      Phoenix   95000

Selected Columns (Name, Age):
       Name  Age
0    Alice   24
1      Bob   27
2  Charlie   22
3    David   32
4      Eve   29

Filtered Rows (Age > 25):
     Name  Age         City  Salary
1    Bob   27  Los Angeles   80000
3  David   32      Houston  110000
4    Eve   29      Phoenix   95000

DataFrame with Bonus column:
       Name  Age         City  Salary    Bonus
0    Alice   24     New York   70000   7000.0
1      Bob   27  Los Angeles   80000   8000.0
2  Charlie   22      Chicago  120000  12000.0
3    David   32      Houston  110000  11000.0
4      Eve   29      Phoenix   95000   9500.0

DataFrame after Dropping Bonus column:
       Name  Age         City  Salary
0    Alice   24     New York   70000
1      Bob   27  Los Angeles   80000
2  Charlie   22      Chicago  120000
3    David   32      Houston  110000
4      Eve   29      Phoenix   95000

Average Salary by City:
 City
Chicago        120000.0
Houston        110000.0
Los Angeles     80000.0
New York        70000.0
Phoenix         95000.0
Name: Salary, dtype: float64

Sorted DataFrame by Salary (Descending):
```

```
       Name  Age         City  Salary    Bonus
2  Charlie   22      Chicago  120000  12000.0
3    David   32      Houston  110000  11000.0
4      Eve   29      Phoenix   95000   9500.0
1      Bob   27  Los Angeles   80000   8000.0
0    Alice   24     New York   70000   7000.0

DataFrame with Missing Value (NaN):
       Name  Age         City    Salary    Bonus
0    Alice   24     New York   70000.0   7000.0
1      Bob   27  Los Angeles   80000.0   8000.0
2  Charlie   22      Chicago       NaN  12000.0
3    David   32      Houston  110000.0  11000.0
4      Eve   29      Phoenix   95000.0   9500.0

DataFrame after Filling Missing Salary with Mean:
       Name  Age         City    Salary    Bonus
0    Alice   24     New York   70000.0   7000.0
1      Bob   27  Los Angeles   80000.0   8000.0
2  Charlie   22      Chicago   88750.0  12000.0
3    David   32      Houston  110000.0  11000.0
4      Eve   29      Phoenix   95000.0   9500.0

Merged DataFrame:
       Name  Age         City  Salary    Bonus Department
0    Alice   24     New York   70000   7000.0         HR
1      Bob   27  Los Angeles   80000   8000.0         IT
2  Charlie   22      Chicago  120000  12000.0    Finance

Concatenated DataFrame (Vertical Stack):
       Name   Age         City    Salary    Bonus Department
0    Alice  24.0     New York   70000.0   7000.0        NaN
1      Bob  27.0  Los Angeles   80000.0   8000.0        NaN
2  Charlie  22.0      Chicago  120000.0  12000.0        NaN
3    David  32.0      Houston  110000.0  11000.0        NaN
4      Eve  29.0      Phoenix   95000.0   9500.0        NaN
5    Alice   NaN          NaN       NaN      NaN         HR
6      Bob   NaN          NaN       NaN      NaN         IT
7  Charlie   NaN          NaN       NaN      NaN    Finance

Pivoted DataFrame (Average Salary by City):
                Salary
City
Chicago      120000.0
Houston      110000.0
Los Angeles   80000.0
New York      70000.0
Phoenix       95000.0
```

EXPERIMENT NO. 04

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Apply and explore multiple types of plots using Matplotlib package

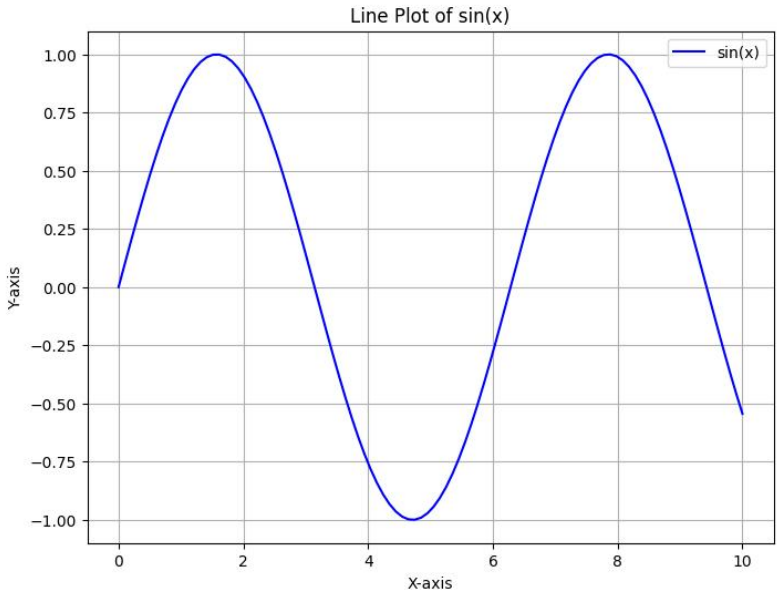
Program:
```
import matplotlib.pyplot as plt
import numpy as np

# Sample data for plotting
x = np.linspace(0, 10, 100)
y = np.sin(x)

# i. Line Plot
plt.figure(figsize=(8, 6))
plt.plot(x, y, label='sin(x)', color='blue')
plt.title('Line Plot of sin(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()

# ii. Scatter Plot
y2 = np.cos(x)
plt.figure(figsize=(8, 6))
plt.scatter(x, y, label='sin(x)', color='red')
plt.scatter(x, y2, label='cos(x)', color='green')
plt.title('Scatter Plot of sin(x) and cos(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.show()

# iii. Bar Plot
categories = ['A', 'B', 'C', 'D', 'E']
values = [3, 7, 2, 5, 6]
plt.figure(figsize=(8, 6))
plt.bar(categories, values, color='purple')
```

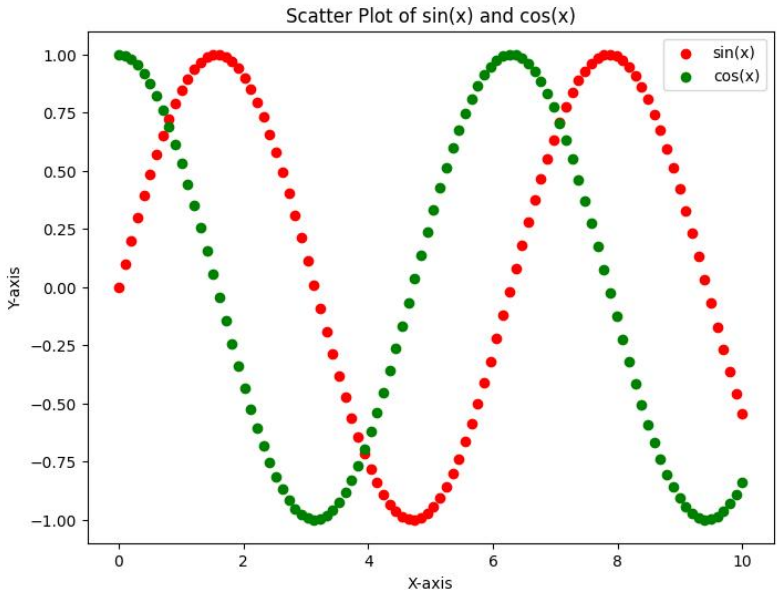
```
plt.title('Bar Plot of Categories')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.show()

# iv. Histogram
data = np.random.randn(1000)
plt.figure(figsize=(8, 6))
plt.hist(data, bins=30, color='orange', edgecolor='black')
plt.title('Histogram of Random Data')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()

# v. Pie Chart
sizes = [30, 25, 45]
labels = ['Python', 'Java', 'C++']
plt.figure(figsize=(8, 6))
plt.pie(sizes,    labels=labels,    autopct='%1.1f%%',    colors=['gold',
'lightcoral', 'lightskyblue'])
plt.title('Pie Chart of Programming Languages')
plt.show()

# vi. Box Plot
np.random.seed(10)
data1 = np.random.normal(0, 1, 100)
data2 = np.random.normal(2, 1.5, 100)
data3 = np.random.normal(4, 2, 100)
plt.figure(figsize=(8, 6))
plt.boxplot([data1, data2, data3], labels=['Data1', 'Data2', 'Data3'])
plt.title('Box Plot of Random Data')
plt.ylabel('Value')
plt.show()

# vii. Heatmap
matrix = np.random.rand(10, 10)
plt.figure(figsize=(8, 6))
plt.imshow(matrix, cmap='viridis', interpolation='nearest')
plt.title('Heatmap of Random Matrix')
plt.colorbar()
plt.show()
```

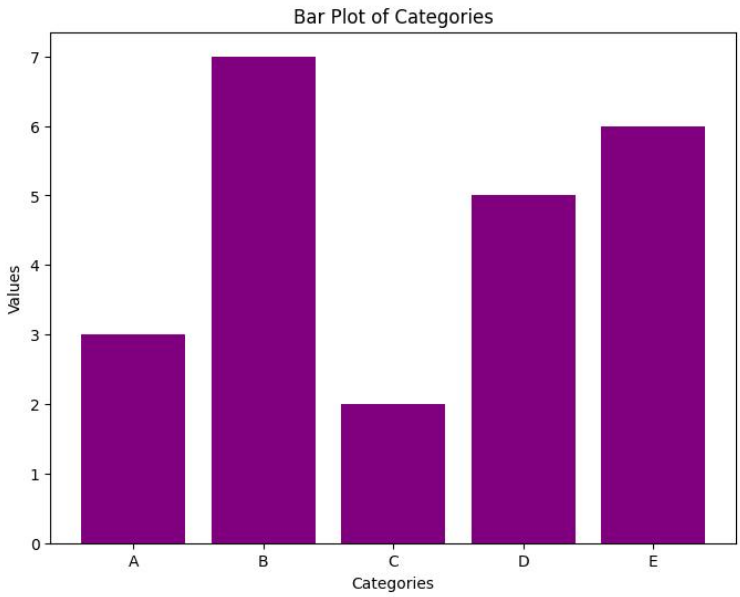
```
# viii. Subplots (Multiple Plots in One Figure)
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

# Line Plot
axs[0, 0].plot(x, y, label='sin(x)', color='blue')
axs[0, 0].set_title('Line Plot')
axs[0, 0].set_xlabel('X-axis')
axs[0, 0].set_ylabel('Y-axis')

# Bar Plot
axs[0, 1].bar(categories, values, color='green')
axs[0, 1].set_title('Bar Plot')
axs[0, 1].set_xlabel('Categories')
axs[0, 1].set_ylabel('Values')

# Scatter Plot
axs[1, 0].scatter(x, y, color='red')
axs[1, 0].set_title('Scatter Plot')
axs[1, 0].set_xlabel('X-axis')
axs[1, 0].set_ylabel('Y-axis')

# Histogram
axs[1, 1].hist(data, bins=30, color='orange', edgecolor='black')
axs[1, 1].set_title('Histogram')
axs[1, 1].set_xlabel('Value')
axs[1, 1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# ix. Area Plot
plt.figure(figsize=(8, 6))
plt.fill_between(x, y, color='skyblue', alpha=0.4)
plt.title('Area Plot of sin(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()

# x. Stack Plot
y_stack = [np.sin(x), np.cos(x), np.tan(x)/10]
plt.figure(figsize=(8, 6))
plt.stackplot(x,   y_stack,   labels=['sin(x)',   'cos(x)',   'tan(x)'],
alpha=0.6)
```

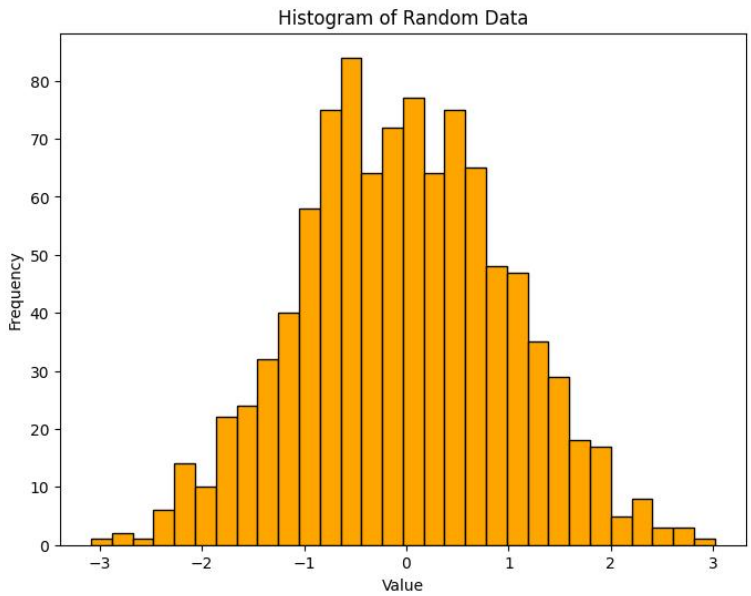
```
plt.title('Stack Plot of sin(x), cos(x), and tan(x)')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend(loc='upper left')
plt.show()
```

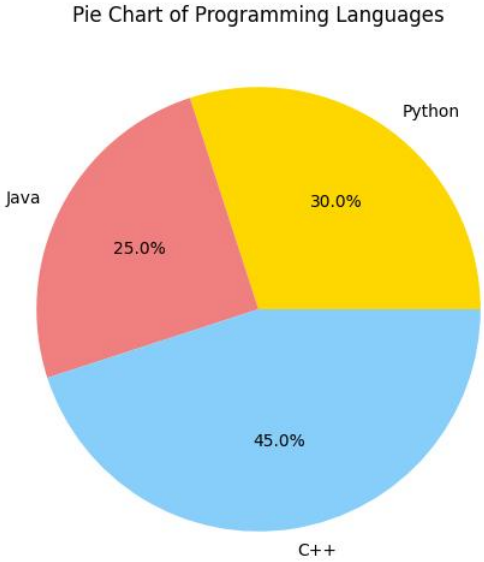
OUTPUT:

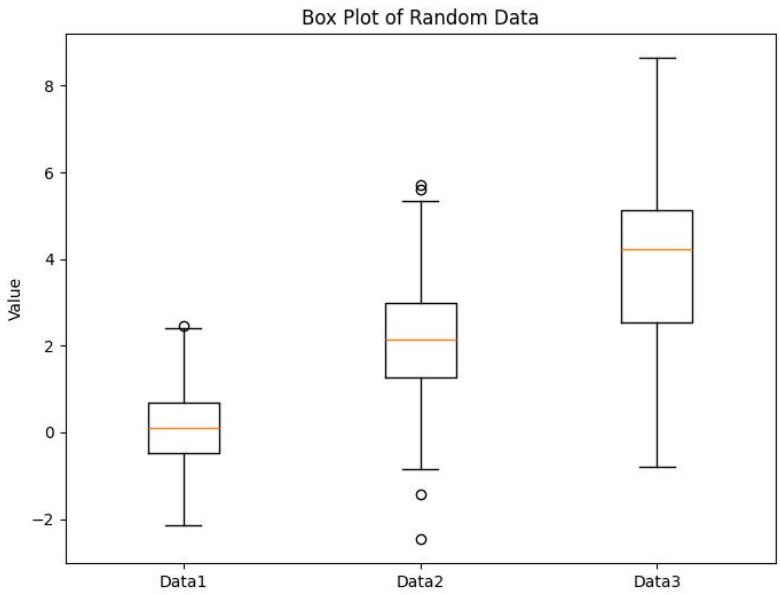


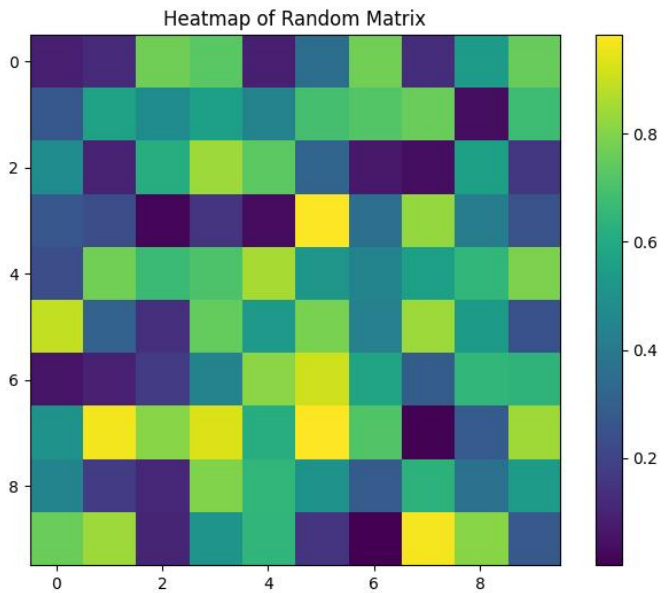


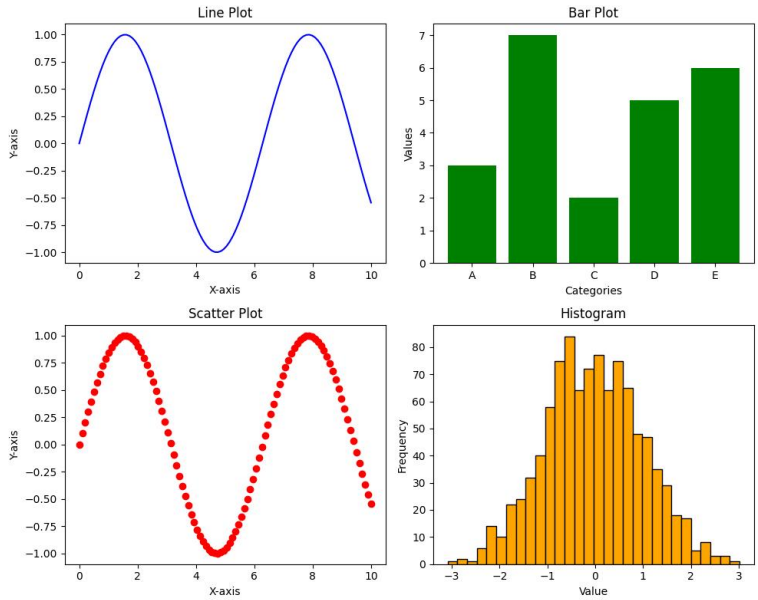

Bar Plot of Categories
Values
Categories
A
B
C
D
E
0
1
2
3
4
5
6
7

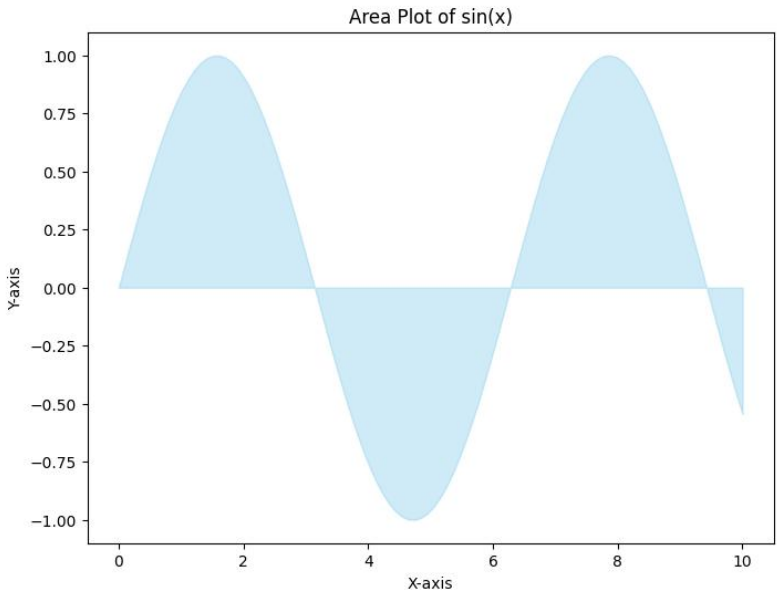

Histogram of Random Data
Frequency
Value
−3
−2
−1
0
1
2
3
0
10
20
30
40
50
60
70
80

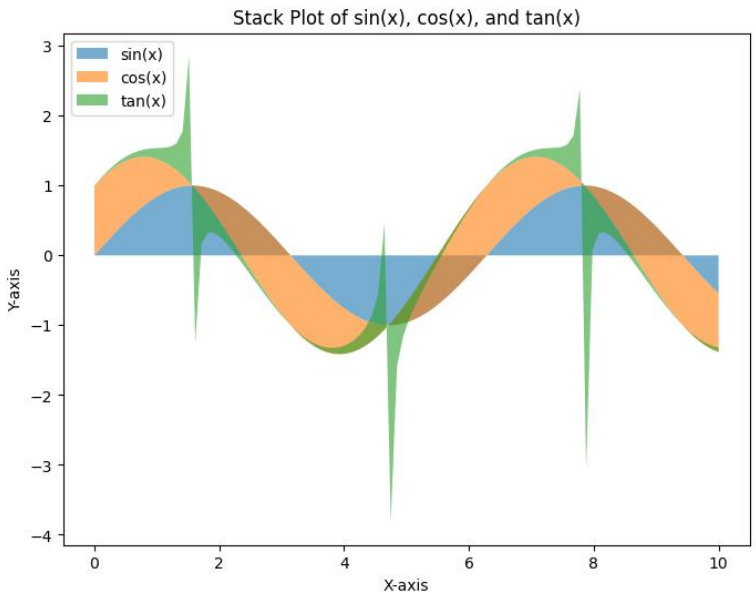

Pie Chart of Programming Languages
Python
30.0%
Java
25.0%
45.0%
C++


Box Plot of Random Data
Value
Data1
Data2
Data3


Heatmap of Random Matrix


Line Plot
Y-axis
X-axis
Bar Plot
Values
Categories
A
B
C
D
E
Scatter Plot
Y-axis
X-axis
Histogram
Frequency
Value


Area Plot of sin(x)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Y-axis
0
2
4
6
8
10
X-axis


Stack Plot of sin(x), cos(x), and tan(x)
sin(x)
cos(x)
tan(x)
3
2
1
0
−1
−2
−3
−4
Y-axis
0
2
4
6
8
10
X-axis

EXPERIMENT NO. 05

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Write a Python Program to import a CSV file and perform various Statistical and Comparison operations on rows/columns.

Program:
```
import pandas as pd

# Load the CSV file generated by the data generation script
file_path = 'exp_05_sample_data.csv'   # Ensure this matches the file
generated by the first script

# Import CSV into a Pandas DataFrame
df = pd.read_csv(file_path)

# i. Display the first few rows of the DataFrame
print("\nFirst 5 rows of the DataFrame:\n", df.head())

# ii. Check for missing values
print("\nMissing Values in Each Column:\n", df.isnull().sum())

# iii. Basic Statistical Summary (mean, std, min, 25%, 50%, 75%, max)
print("\nStatistical Summary of the DataFrame:\n", df.describe())

# iv. Comparison operation - Find rows where a specific condition is
met
# Example: Find rows where Age is greater than 30
filtered_data = df[df['Age'] > 30]
print("\nRows where Age > 30:\n", filtered_data)

# v. Comparison of two columns (e.g., compare 'Salary' and 'Age')
df['Salary_vs_Age'] = df['Salary'] > df['Age']
print("\nComparison of Salary and Age (Salary > Age):\n", df[['Name',
'Age', 'Salary', 'Salary_vs_Age']])

# vi. Grouping data by a categorical column and calculating aggregate
functions (e.g., mean salary by 'City')
grouped_data = df.groupby('City')['Salary'].mean()
```

```
print("\nAverage Salary by City:\n", grouped_data)

# vii. Sort the data by a column (e.g., Sort by 'Salary' in descending order)
sorted_data = df.sort_values(by='Salary', ascending=False)
print("\nData Sorted by Salary (Descending):\n", sorted_data)

# viii. Correlation between numerical columns (e.g., 'Age' and 'Salary')
correlation = df[['Age', 'Salary']].corr()
print("\nCorrelation between Age and Salary:\n", correlation)

# ix. Create a new column with the result of a mathematical operation (e.g., Salary to Age ratio)
df['Salary_to_Age_Ratio'] = df['Salary'] / df['Age']
print("\nDataFrame with Salary to Age Ratio:\n", df[['Name', 'Age', 'Salary', 'Salary_to_Age_Ratio']])

# x. Drop rows with missing values (if any)
df_dropped_na = df.dropna()
print("\nDataFrame after Dropping Rows with Missing Values:\n", df_dropped_na)

# xi. Fill missing values (if any) with a constant or method (e.g., fill NaN with the column mean)
# Apply fillna only to numeric columns
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
df_filled_na = df.copy()

# Fill NaN values in numeric columns with their respective column mean
df_filled_na[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())

print("\nDataFrame after Filling Missing Values with Column Mean:\n", df_filled_na)

# xii. Apply a custom function to a column (e.g., Apply function to convert Age to category)
df['Age_Category'] = df['Age'].apply(lambda x: 'Young' if x < 30 else 'Old')
print("\nDataFrame with Age Category:\n", df[['Name', 'Age', 'Age_Category']])
```

OUTPUT:

```
First 5 rows of the DataFrame:
       Name  Age         City  Salary
0    Alice   26      Chicago   78693
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
3    David   30      Houston   75658
4      Eve   27  Los Angeles  134478

Missing Values in Each Column:
 Name      0
Age       0
City      0
Salary    0
dtype: int64

Statistical Summary of the DataFrame:
              Age         Salary
count  10.000000      10.000000
mean   30.300000  103448.400000
std     5.271517   31987.972674
min    23.000000   52747.000000
25%    26.250000   76416.750000
50%    30.000000  112437.500000
75%    33.000000  131341.500000
max    39.000000  143016.000000

Rows where Age > 30:
       Name  Age         City  Salary
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
6    Grace   38      Phoenix   52747

Comparison of Salary and Age (Salary > Age):
       Name  Age  Salary  Salary_vs_Age
0    Alice   26   78693           True
1      Bob   39  121932           True
2  Charlie   34  143016           True
3    David   30   75658           True
4      Eve   27  134478           True
5    Frank   26   68431           True
6    Grace   38   52747           True
7   Hannah   30  109150           True
8      Ian   30  115725           True
9     Jack   23  134654           True

Average Salary by City:
 City
Chicago         78693.000000
```

```
Houston         86604.666667
Los Angeles    137382.666667
New York       109150.000000
Phoenix         87339.500000
Name: Salary, dtype: float64

Data Sorted by Salary (Descending):
       Name  Age         City  Salary  Salary_vs_Age
2  Charlie   34  Los Angeles  143016           True
9     Jack   23  Los Angeles  134654           True
4      Eve   27  Los Angeles  134478           True
1      Bob   39      Phoenix  121932           True
8      Ian   30      Houston  115725           True
7   Hannah   30     New York  109150           True
0    Alice   26      Chicago   78693           True
3    David   30      Houston   75658           True
5    Frank   26      Houston   68431           True
6    Grace   38      Phoenix   52747           True

Correlation between Age and Salary:
              Age    Salary
Age     1.000000 -0.101096
Salary -0.101096  1.000000

DataFrame with Salary to Age Ratio:
       Name  Age  Salary  Salary_to_Age_Ratio
0    Alice   26   78693          3026.653846
1      Bob   39  121932          3126.461538
2  Charlie   34  143016          4206.352941
3    David   30   75658          2521.933333
4      Eve   27  134478          4980.666667
5    Frank   26   68431          2631.961538
6    Grace   38   52747          1388.078947
7   Hannah   30  109150          3638.333333
8      Ian   30  115725          3857.500000
9     Jack   23  134654          5854.521739

DataFrame after Dropping Rows with Missing Values:
       Name     Age                     City     Salary     Salary_vs_Age
Salary_to_Age_Ratio
0       Alice      26           Chicago      78693                    True
3026.653846
1           Bob      39           Phoenix    121932                    True
3126.461538
2    Charlie       34    Los  Angeles    143016                    True
4206.352941
3       David      30           Houston      75658                    True
2521.933333
4           Eve      27    Los  Angeles    134478                    True
4980.666667
```

```
5       Frank      26           Houston      68431                    True
2631.961538
6       Grace      38           Phoenix      52747                    True
1388.078947
7      Hannah      30          New  York    109150                    True
3638.333333
8           Ian      30           Houston    115725                    True
3857.500000
9         Jack      23    Los  Angeles    134654                    True
5854.521739

DataFrame after Filling Missing Values with Column Mean:
       Name     Age                      City     Salary     Salary_vs_Age
Salary_to_Age_Ratio
0       Alice      26           Chicago      78693                    True
3026.653846
1           Bob      39           Phoenix    121932                    True
3126.461538
2    Charlie      34    Los  Angeles    143016                    True
4206.352941
3       David      30           Houston      75658                    True
2521.933333
4           Eve      27    Los  Angeles    134478                    True
4980.666667
5       Frank      26           Houston      68431                    True
2631.961538
6       Grace      38           Phoenix      52747                    True
1388.078947
7      Hannah      30          New  York    109150                    True
3638.333333
8           Ian      30           Houston    115725                    True
3857.500000
9         Jack      23    Los  Angeles    134654                    True
5854.521739

DataFrame with Age Category:
       Name  Age Age_Category
0    Alice   26        Young
1      Bob   39          Old
2  Charlie   34          Old
3    David   30          Old
4      Eve   27        Young
5    Frank   26        Young
6    Grace   38          Old
7   Hannah   30          Old
8      Ian   30          Old
9     Jack   23        Young
```

EXPERIMENT NO. 06

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Write a Python Program to import any CSV file to Pandas DataFrame and perform the following:
i. Visualize the first and last 10 records
ii. Get the shape, index and column details
iii. Select/Delete the records(rows)/columns based on conditions.
iv. Perform ranking and sorting operations.
v. Do required statistical operations on the given columns.
vi. Find the count and uniqueness of the given categorical values.
vii. Rename single/multiple columns.

Program:
```
import pandas as pd

# Step 1: Import the CSV file into a Pandas DataFrame
file_path = 'test_data.csv'  # Replace with the path of your CSV file
df = pd.read_csv(file_path)

# i. Visualize the first and last 10 records
print("\nFirst 10 records:\n", df.head(10))
print("\nLast 10 records:\n", df.tail(10))

# ii. Get the shape, index, and column details
print("\nShape of the DataFrame:", df.shape)  # (rows, columns)
print("\nIndex details:\n", df.index)
print("\nColumn details:\n", df.columns)

# iii. Select/Delete the records (rows)/columns based on conditions
# Select rows where Age > 30
selected_rows = df[df['Age'] > 30]
print("\nRows where Age > 30:\n", selected_rows)

# Delete rows where Salary < 60000
df_dropped_rows = df[df['Salary'] >= 60000]
print("\nData after deleting rows where Salary < 60000:\n",
df_dropped_rows)
```

```python
# Delete a column 'City'
df_dropped_column = df.drop('City', axis=1)
print("\nData after deleting 'City' column:\n", df_dropped_column)

# iv. Perform ranking and sorting operations
# Sort by Salary in descending order
sorted_df = df.sort_values(by='Salary', ascending=False)
print("\nData sorted by Salary in descending order:\n", sorted_df)

# Rank the salaries
df['Salary_Rank'] = df['Salary'].rank(ascending=False)
print("\nData with Salary Rank:\n", df[['Name', 'Salary', 'Salary_Rank']])

# v. Perform statistical operations on the given columns
# Mean, median, and standard deviation of Salary and Age columns
salary_mean = df['Salary'].mean()
age_median = df['Age'].median()
salary_std = df['Salary'].std()

print(f"\nMean Salary: {salary_mean}")
print(f"Median Age: {age_median}")
print(f"Salary Standard Deviation: {salary_std}")

# vi. Find the count and uniqueness of the given categorical values (e.g., 'City')
city_count = df['City'].value_counts()
city_unique = df['City'].unique()

print("\nCount of each unique City:\n", city_count)
print("\nUnique Cities:\n", city_unique)

# vii. Rename single/multiple columns
df_renamed = df.rename(columns={'Name': 'Full_Name', 'Age': 'Age_Years'})
print("\nData with renamed columns (Name -> Full_Name, Age -> Age_Years):\n", df_renamed)
```

OUTPUT:

```
First 10 records:
       Name  Age         City  Salary
0    Alice   26      Chicago   78693
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
3    David   30      Houston   75658
4      Eve   27  Los Angeles  134478
5    Frank   26      Houston   68431
6    Grace   38      Phoenix   52747
7   Hannah   30     New York  109150
8      Ian   30      Houston  115725
9     Jack   23  Los Angeles  134654

Last 10 records:
       Name  Age         City  Salary
0    Alice   26      Chicago   78693
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
3    David   30      Houston   75658
4      Eve   27  Los Angeles  134478
5    Frank   26      Houston   68431
6    Grace   38      Phoenix   52747
7   Hannah   30     New York  109150
8      Ian   30      Houston  115725
9     Jack   23  Los Angeles  134654

Shape of the DataFrame: (10, 4)

Index details:
 RangeIndex(start=0, stop=10, step=1)

Column details:
 Index(['Name', 'Age', 'City', 'Salary'], dtype='object')

Rows where Age > 30:
       Name  Age         City  Salary
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
6    Grace   38      Phoenix   52747

Data after deleting rows where Salary < 60000:
       Name  Age         City  Salary
0    Alice   26      Chicago   78693
1      Bob   39      Phoenix  121932
2  Charlie   34  Los Angeles  143016
3    David   30      Houston   75658
4      Eve   27  Los Angeles  134478
5    Frank   26      Houston   68431
```

```
7   Hannah   30     New York  109150
8      Ian   30      Houston  115725
9     Jack   23  Los Angeles  134654

Data after deleting 'City' column:
       Name  Age  Salary
0    Alice   26   78693
1      Bob   39  121932
2  Charlie   34  143016
3    David   30   75658
4      Eve   27  134478
5    Frank   26   68431
6    Grace   38   52747
7   Hannah   30  109150
8      Ian   30  115725
9     Jack   23  134654

Data sorted by Salary in descending order:
       Name  Age         City  Salary
2  Charlie   34  Los Angeles  143016
9     Jack   23  Los Angeles  134654
4      Eve   27  Los Angeles  134478
1      Bob   39      Phoenix  121932
8      Ian   30      Houston  115725
7   Hannah   30     New York  109150
0    Alice   26      Chicago   78693
3    David   30      Houston   75658
5    Frank   26      Houston   68431
6    Grace   38      Phoenix   52747

Data with Salary Rank:
       Name  Salary  Salary_Rank
0    Alice   78693          7.0
1      Bob  121932          4.0
2  Charlie  143016          1.0
3    David   75658          8.0
4      Eve  134478          3.0
5    Frank   68431          9.0
6    Grace   52747         10.0
7   Hannah  109150          6.0
8      Ian  115725          5.0
9     Jack  134654          2.0

Mean Salary: 103448.4
Median Age: 30.0
Salary Standard Deviation: 31987.972674463483

Count of each unique City:
 City
Los Angeles    3
Houston        3
```

```
Phoenix        2
Chicago        1
New York       1
Name: count, dtype: int64

Unique Cities:
 ['Chicago' 'Phoenix' 'Los Angeles' 'Houston' 'New York']

Data with renamed columns (Name -> Full_Name, Age -> Age_Years):
   Full_Name  Age_Years         City  Salary  Salary_Rank
0      Alice         26      Chicago   78693          7.0
1        Bob         39      Phoenix  121932          4.0
2    Charlie         34  Los Angeles  143016          1.0
3      David         30      Houston   75658          8.0
4        Eve         27  Los Angeles  134478          3.0
5      Frank         26      Houston   68431          9.0
6      Grace         38      Phoenix   52747         10.0
7     Hannah         30     New York  109150          6.0
8        Ian         30      Houston  115725          5.0
9       Jack         23  Los Angeles  134654          2.0
```

EXPERIMENT NO. 07

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Write a Python Program to import any CSV file to Pandas DataFrame and perform the following:
i. Handle missing data by detecting and dropping/ filling missing values.
ii. Transform data using different methods.
iii. Detect and filter outliers.
iv. Perform Vectorized String operations on Pandas Series.
v. Visualize data using Line Plots, Bar Plots, Histograms, Density Plots and Scatter Plots.

Program:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Import the CSV file into a Pandas DataFrame
file_path = 'test_data_with_missing.csv'   # Replace with the path of
your CSV file
df = pd.read_csv(file_path)

# i. Handle missing data by detecting and dropping/filling missing
values
print("\nMissing data in each column:\n", df.isnull().sum())

# Dropping rows with missing values
df_dropped = df.dropna()
print("\nData after dropping rows with missing values:\n", df_dropped)

# Filling missing values with the column mean (only for numeric columns)
df_filled = df.copy()
df_filled['Age'] = df_filled['Age'].fillna(df['Age'].mean())
df_filled['Salary'] = df_filled['Salary'].fillna(df['Salary'].mean())
print("\nData after filling missing values with column means:\n",
df_filled)

# ii. Transform data using different methods
# Example: Log transformation of Salary to normalize
```

```
df_filled['Log_Salary'] = np.log(df_filled['Salary'])
print("\nData after log transformation on Salary:\n", df_filled[['Name',
'Salary', 'Log_Salary']])

# iii. Detect and filter outliers
# Using the Z-score method to detect outliers in 'Salary'
from scipy.stats import zscore

df_filled['Salary_Zscore'] = zscore(df_filled['Salary'])
print("\nZ-scores of Salary:\n", df_filled[['Name', 'Salary',
'Salary_Zscore']])

# Filter rows with Z-score > 3 (outliers)
df_no_outliers = df_filled[df_filled['Salary_Zscore'].abs() <= 3]
print("\nData after filtering out outliers (Z-score <= 3):\n",
df_no_outliers)

# iv. Perform Vectorized String operations on Pandas Series
# Example: Convert all names to uppercase
df_filled['Name_Upper'] = df_filled['Name'].str.upper()
print("\nData after applying uppercase operation on 'Name' column:\n",
df_filled[['Name', 'Name_Upper']])

# v. Visualize data using Line Plots, Bar Plots, Histograms, Density
Plots, and Scatter Plots
# Line Plot - Plot Salary vs. Age
plt.figure(figsize=(8, 5))
plt.plot(df_filled['Age'], df_filled['Salary'], marker='o',
linestyle='-', color='b')
plt.title('Salary vs. Age (Line Plot)')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.grid(True)
plt.show()

# Bar Plot - Bar plot of the count of individuals in each city
plt.figure(figsize=(8, 5))
df_filled['City'].value_counts().plot(kind='bar', color='skyblue')
plt.title('Count of Individuals by City')
plt.xlabel('City')
plt.ylabel('Count')
plt.show()
```

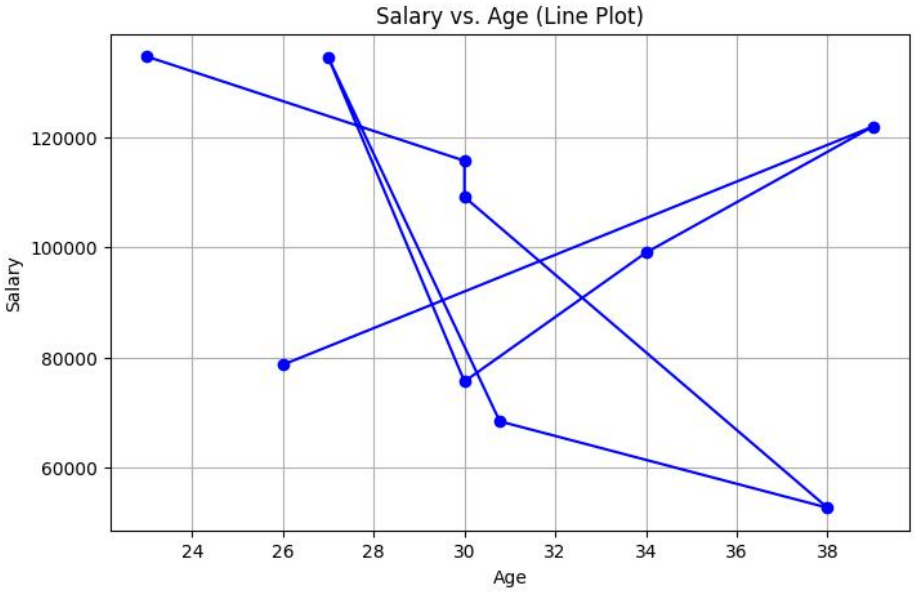
```python
# Histogram - Distribution of Salaries
plt.figure(figsize=(8, 5))
df_filled['Salary'].hist(bins=10, color='orange', edgecolor='black')
plt.title('Distribution of Salaries (Histogram)')
plt.xlabel('Salary')
plt.ylabel('Frequency')
plt.show()

# Density Plot - Density plot of Age
plt.figure(figsize=(8, 5))
sns.kdeplot(df_filled['Age'], shade=True, color='g')
plt.title('Density Plot of Age')
plt.xlabel('Age')
plt.ylabel('Density')
plt.show()

# Scatter Plot - Salary vs. Age
plt.figure(figsize=(8, 5))
plt.scatter(df_filled['Age'], df_filled['Salary'], color='red')
plt.title('Salary vs. Age (Scatter Plot)')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.show()
```

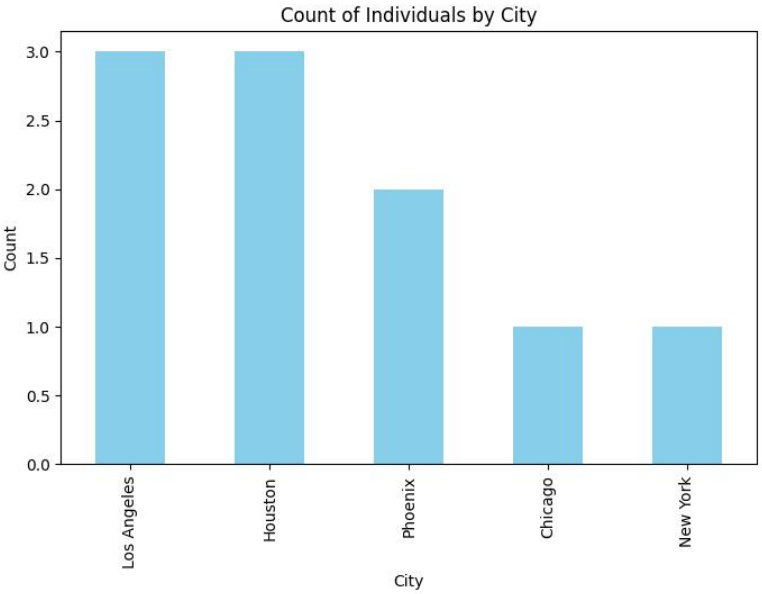
OUTPUT:

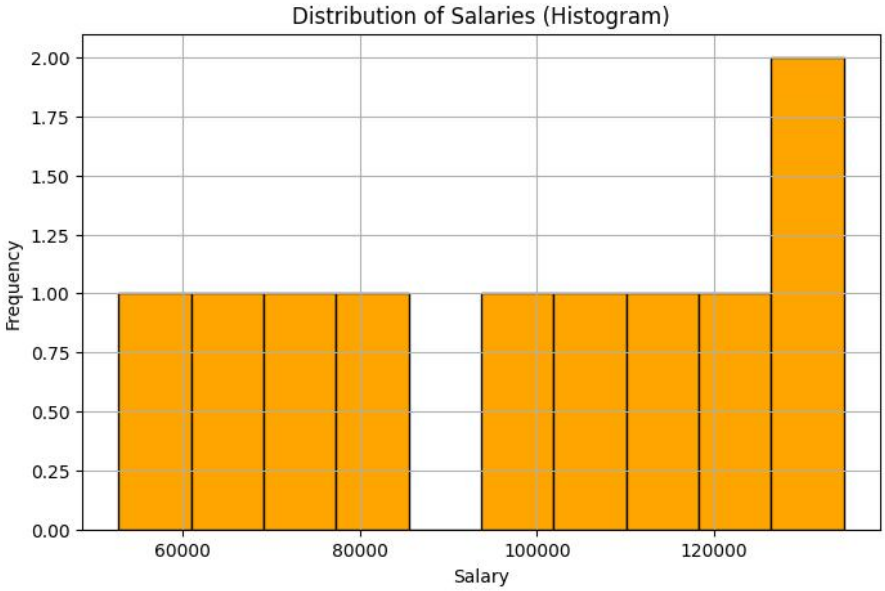
```
Missing data in each column:
 Name      0
Age       1
City      0
Salary    1
dtype: int64

Data after dropping rows with missing values:
      Name   Age         City    Salary
0    Alice  26.0      Chicago   78693.0
1      Bob  39.0      Phoenix  121932.0
3    David  30.0      Houston   75658.0
4      Eve  27.0  Los Angeles  134478.0
6    Grace  38.0      Phoenix   52747.0
7   Hannah  30.0     New York  109150.0
8      Ian  30.0      Houston  115725.0
9     Jack  23.0  Los Angeles  134654.0

Data after filling missing values with column means:
```

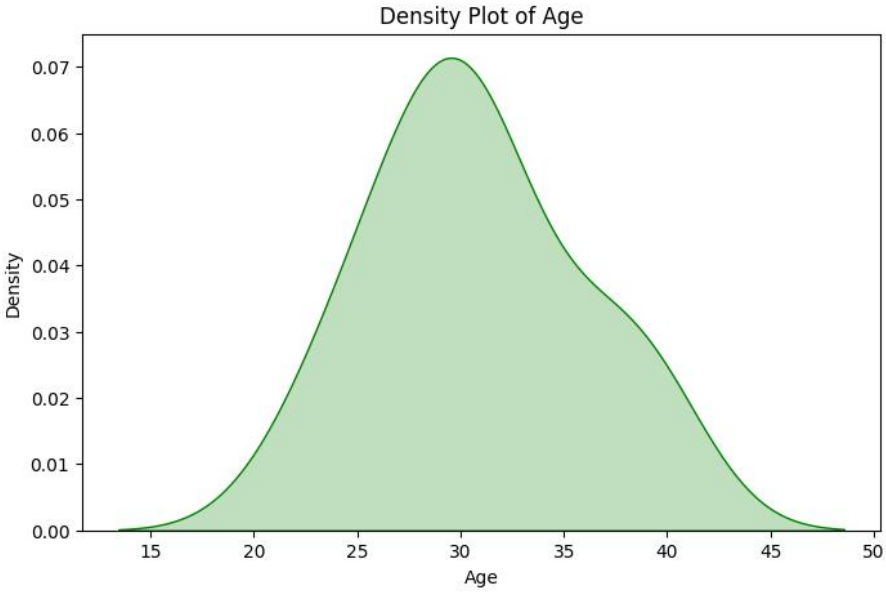
```
       Name        Age         City    Salary
0    Alice  26.000000      Chicago   78693.0
1      Bob  39.000000      Phoenix  121932.0
2  Charlie  34.000000  Los Angeles   99052.0
3    David  30.000000      Houston   75658.0
4      Eve  27.000000  Los Angeles  134478.0
5    Frank  30.777778      Houston   68431.0
6    Grace  38.000000      Phoenix   52747.0
7   Hannah  30.000000     New York  109150.0
8      Ian  30.000000      Houston  115725.0
9     Jack  23.000000  Los Angeles  134654.0

Data after log transformation on Salary:
       Name    Salary  Log_Salary
0    Alice   78693.0   11.273309
1      Bob  121932.0   11.711219
2  Charlie   99052.0   11.503400
3    David   75658.0   11.233978
4      Eve  134478.0   11.809156
5    Frank   68431.0   11.133581
6    Grace   52747.0   10.873262
7   Hannah  109150.0   11.600478
8      Ian  115725.0   11.658972
9     Jack  134654.0   11.810464

Z-scores of Salary:
       Name    Salary  Salary_Zscore
0    Alice   78693.0      -0.744921
1      Bob  121932.0       0.837162
2  Charlie   99052.0       0.000000
3    David   75658.0      -0.855969
4      Eve  134478.0       1.296211
5    Frank   68431.0      -1.120400
6    Grace   52747.0      -1.694265
7   Hannah  109150.0       0.369478
8      Ian  115725.0       0.610053
9     Jack  134654.0       1.302651

Data after filtering out outliers (Z-score <= 3):
       Name             Age               City       Salary   Log_Salary
Salary_Zscore
0    Alice  26.000000      Chicago   78693.0   11.273309      -0.744921
1      Bob  39.000000      Phoenix  121932.0   11.711219       0.837162
2  Charlie  34.000000  Los Angeles   99052.0   11.503400       0.000000
3    David  30.000000      Houston   75658.0   11.233978      -0.855969
4      Eve  27.000000  Los Angeles  134478.0   11.809156       1.296211
5    Frank  30.777778      Houston   68431.0   11.133581      -1.120400
6    Grace  38.000000      Phoenix   52747.0   10.873262      -1.694265
7   Hannah  30.000000     New York  109150.0   11.600478       0.369478
8      Ian  30.000000      Houston  115725.0   11.658972       0.610053
9     Jack  23.000000  Los Angeles  134654.0   11.810464       1.302651
```

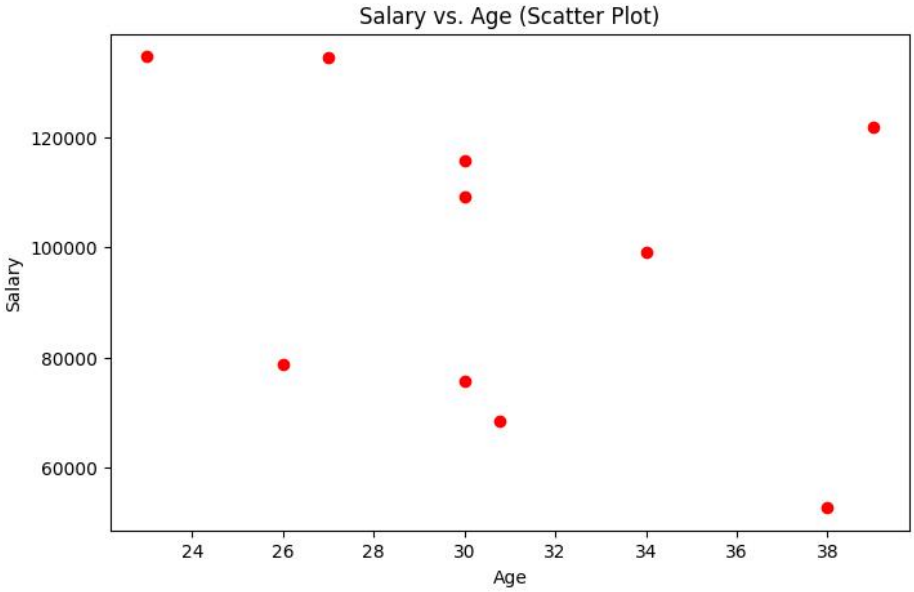
```
Data after applying uppercase operation on 'Name' column:
       Name Name_Upper
0    Alice      ALICE
1      Bob        BOB
2  Charlie    CHARLIE
3    David      DAVID
4      Eve        EVE
5    Frank      FRANK
6    Grace      GRACE
7   Hannah     HANNAH
8      Ian        IAN
9     Jack       JACK
```






Distribution of Salaries (Histogram)
Frequency
Salary
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
60000
80000
100000
120000


Density Plot of Age
Density
Age
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00
15
20
25
30
35
40
45
50


Salary vs. Age (Scatter Plot)
Salary
Age
120000
100000
80000
60000
24
26
28
30
32
34
36
38

EXPERIMENT NO. 08

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

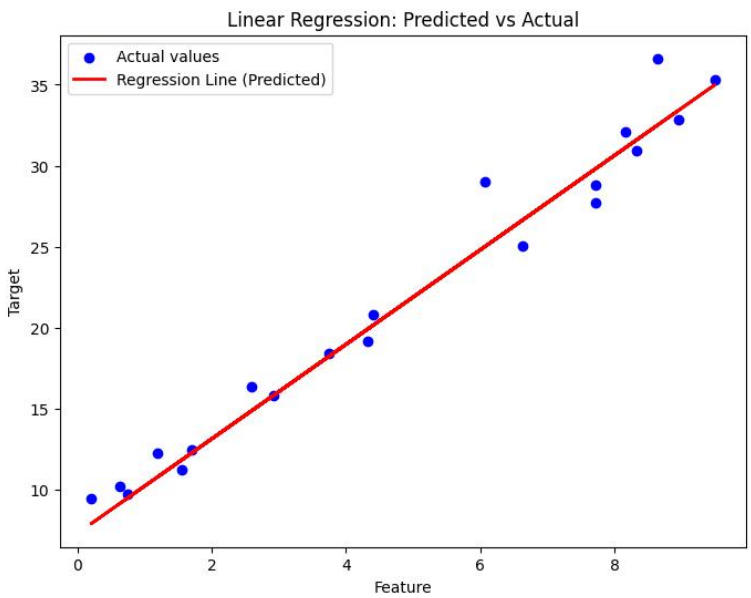
Problem Statement:
Write a Python Program to use the scikit-learn package in python to implement the regression model and its related methods.

Program:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from  sklearn.metrics  import  mean_absolute_error,  mean_squared_error,
r2_score

# Load the dataset
file_path = 'regression_data.csv'  # Path to the generated CSV file
df = pd.read_csv(file_path)

# Step 1: Prepare the data
X = df[['Feature']]  # Feature
y = df['Target']    # Target

# Step 2: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Step 3: Create and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 4: Make predictions
y_pred = model.predict(X_test)

# Step 5: Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Evaluation Metrics:")
print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared: {r2}")

# Step 6: Visualize the results (Predicted vs Actual)
plt.figure(figsize=(8, 6))
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression
Line (Predicted)')
plt.title('Linear Regression: Predicted vs Actual')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.legend()
plt.show()
```

OUTPUT:

```
Model Evaluation Metrics:
Mean Absolute Error (MAE): 1.182685155837957
Mean Squared Error (MSE): 2.6147980548680128
R-squared: 0.9686345415489115
```



EXPERIMENT NO. 09

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Use publicly available medical dataset and perform univariate and bivariate analysis and compare results of above analysis for two datasets.

Program:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
import requests
import warnings
warnings.filterwarnings('ignore')

print("=== Medical Dataset Analysis: Univariate and Bivariate Analysis ===\n")

# ========== Dataset 1: Breast Cancer Wisconsin ==========
print("  Loading Dataset 1: Breast Cancer Wisconsin Dataset")
print("-" * 60)

# Load breast cancer dataset from sklearn
breast_cancer = load_breast_cancer()
df1 = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
df1['diagnosis'] = breast_cancer.target

# Map target values to meaningful labels
df1['diagnosis_label'] = df1['diagnosis'].map({0: 'Malignant', 1: 'Benign'})

print("Dataset 1 Info:")
print(f"Shape: {df1.shape}")
print(f"Features: {len(df1.columns)-2} medical measurements")
print(f"Target: Diagnosis (Malignant/Benign)")
```

```
print("\nFirst 5 rows:")
print(df1[['mean radius', 'mean texture', 'mean perimeter', 'mean area',
'diagnosis_label']].head())

# ========================= Dataset 2: Heart Disease Dataset
=========================
print("\n Loading Dataset 2: Heart Disease Dataset")
print("-" * 60)

# Download Heart Disease dataset from UCI
try:
    url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/heart-disease/processed.cleveland.data"
    column_names = [
        'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
        'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'
    ]

    df2 = pd.read_csv(url, names=column_names)

    # Clean the data
    df2 = df2.replace('?', np.nan)
    df2 = df2.dropna()

    # Convert to appropriate data types
    for col in df2.columns:
        df2[col] = pd.to_numeric(df2[col])

    # Binary classification: 0 = no disease, 1-4 = disease present
    df2['heart_disease'] = (df2['target'] > 0).astype(int)
    df2['heart_disease_label'] = df2['heart_disease'].map({0: 'No
Disease', 1: 'Disease Present'})

    print("Dataset 2 Info:")
    print(f"Shape: {df2.shape}")
    print(f"Features: Age, Cholesterol, Blood Pressure, etc.")
    print(f"Target: Heart Disease (Present/Absent)")
    print("\nFirst 5 rows:")
    print(df2[['age', 'chol', 'trestbps', 'thalach',
'heart_disease_label']].head())

except Exception as e:
    print(f"Error loading heart disease dataset: {e}")
```

```
    print("Creating alternative synthetic heart disease data...")

    # Create synthetic heart disease data as backup
    np.random.seed(42)
    n_samples = 300

    df2 = pd.DataFrame({
        'age': np.random.normal(54, 9, n_samples).astype(int),
        'chol': np.random.normal(246, 50, n_samples).astype(int),
        'trestbps': np.random.normal(131, 17, n_samples).astype(int),
        'thalach': np.random.normal(150, 23, n_samples).astype(int),
        'heart_disease': np.random.binomial(1, 0.54, n_samples)
    })

    # Ensure realistic ranges
    df2['age'] = np.clip(df2['age'], 29, 77)
    df2['chol'] = np.clip(df2['chol'], 126, 564)
    df2['trestbps'] = np.clip(df2['trestbps'], 94, 200)
    df2['thalach'] = np.clip(df2['thalach'], 71, 202)

    df2['heart_disease_label']    =    df2['heart_disease'].map({0:    'No
Disease', 1: 'Disease Present'})

    print("Synthetic Dataset 2 created successfully!")

#          ==========================          UNIVARIATE          ANALYSIS
==========================
print("\n" + "="*80)
print("  UNIVARIATE ANALYSIS")
print("="*80)

# Create figure for univariate analysis
fig, axes = plt.subplots(3, 4, figsize=(20, 15))
fig.suptitle('Univariate  Analysis:  Distribution  of  Key  Variables',
fontsize=16, fontweight='bold')

# Dataset 1 - Breast Cancer
print("\n  Dataset 1 - Breast Cancer: Univariate Analysis")
print("-" * 50)

# Select key features for analysis
key_features_bc  =  ['mean  radius',  'mean  texture',  'mean  area',  'mean
smoothness']
```

```
for i, feature in enumerate(key_features_bc):
    # Histogram
    axes[0, i].hist(df1[feature], bins=30, alpha=0.7, color='skyblue',
edgecolor='black')
    axes[0, i].set_title(f'BC Dataset: {feature}')
    axes[0, i].set_xlabel(feature)
    axes[0, i].set_ylabel('Frequency')
    axes[0, i].grid(True, alpha=0.3)

    # Print statistics
    print(f"\n{feature.title()}:")
    print(f"  Mean: {df1[feature].mean():.3f}")
    print(f"  Std:  {df1[feature].std():.3f}")
    print(f"  Min:  {df1[feature].min():.3f}")
    print(f"  Max:  {df1[feature].max():.3f}")

# Dataset 2 - Heart Disease
print("\n  Dataset 2 - Heart Disease: Univariate Analysis")
print("-" * 50)

key_features_hd = ['age', 'chol', 'trestbps', 'thalach']

for i, feature in enumerate(key_features_hd):
    # Histogram
    axes[1,        i].hist(df2[feature],        bins=30,        alpha=0.7,
color='lightcoral', edgecolor='black')
    axes[1, i].set_title(f'Heart Disease Dataset: {feature}')
    axes[1, i].set_xlabel(feature)
    axes[1, i].set_ylabel('Frequency')
    axes[1, i].grid(True, alpha=0.3)

    # Print statistics
    print(f"\n{feature.title()}:")
    print(f"  Mean: {df2[feature].mean():.3f}")
    print(f"  Std:  {df2[feature].std():.3f}")
    print(f"  Min:  {df2[feature].min():.3f}")
    print(f"  Max:  {df2[feature].max():.3f}")

# Target variable distribution
axes[2, 0].bar(df1['diagnosis_label'].value_counts().index,
               df1['diagnosis_label'].value_counts().values,
               color=['lightblue', 'lightgreen'], alpha=0.8)
```

```
axes[2, 0].set_title('Breast Cancer: Diagnosis Distribution')
axes[2, 0].set_ylabel('Count')

axes[2, 1].bar(df2['heart_disease_label'].value_counts().index,
               df2['heart_disease_label'].value_counts().values,
               color=['lightcoral', 'lightyellow'], alpha=0.8)
axes[2, 1].set_title('Heart Disease: Disease Distribution')
axes[2, 1].set_ylabel('Count')

# Box plots for comparison
axes[2, 2].boxplot([df1['mean radius'], df2['age']], labels=['BC: Mean
Radius', 'HD: Age'])
axes[2, 2].set_title('Box Plot Comparison')
axes[2, 2].set_ylabel('Values')

# Density plots
axes[2, 3].hist(df1['mean area'], bins=30, alpha=0.5, label='BC: Mean
Area', color='blue', density=True)
axes[2, 3].hist(df2['chol'], bins=30, alpha=0.5, label='HD:
Cholesterol', color='red', density=True)
axes[2, 3].set_title('Density Comparison')
axes[2, 3].legend()
axes[2, 3].set_ylabel('Density')

plt.tight_layout()
plt.savefig('univariate_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

# ========== BIVARIATE ANALYSIS
==========
print("\n" + "="*80)
print("  BIVARIATE ANALYSIS")
print("="*80)

# Create figure for bivariate analysis
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Bivariate Analysis: Relationships Between Variables',
fontsize=16, fontweight='bold')

print("\n  Dataset 1 - Breast Cancer: Bivariate Analysis")
print("-" * 50)

# Scatter plot: Mean radius vs Mean area colored by diagnosis
```

```
scatter1 = axes[0, 0].scatter(df1['mean radius'], df1['mean area'],
                              c=df1['diagnosis'],        cmap='coolwarm',
alpha=0.6)
axes[0, 0].set_xlabel('Mean Radius')
axes[0, 0].set_ylabel('Mean Area')
axes[0, 0].set_title('BC: Mean Radius vs Mean Area')
plt.colorbar(scatter1,  ax=axes[0,  0],  label='Diagnosis  (0=Malignant,
1=Benign)')

# Correlation analysis
correlation_bc = df1[['mean radius', 'mean texture', 'mean area', 'mean
smoothness']].corr()
print("\nCorrelation Matrix - Breast Cancer:")
print(correlation_bc.round(3))

# Box plot: Mean texture by diagnosis
df1.boxplot(column='mean texture', by='diagnosis_label', ax=axes[0, 1])
axes[0, 1].set_title('BC: Mean Texture by Diagnosis')
axes[0, 1].set_xlabel('Diagnosis')
axes[0, 1].set_ylabel('Mean Texture')

# Heatmap of correlation
im1 = axes[0, 2].imshow(correlation_bc, cmap='coolwarm', aspect='auto')
axes[0, 2].set_xticks(range(len(correlation_bc.columns)))
axes[0, 2].set_yticks(range(len(correlation_bc.columns)))
axes[0, 2].set_xticklabels(correlation_bc.columns, rotation=45)
axes[0, 2].set_yticklabels(correlation_bc.columns)
axes[0, 2].set_title('BC: Correlation Heatmap')
plt.colorbar(im1, ax=axes[0, 2])

print("\n  Dataset 2 - Heart Disease: Bivariate Analysis")
print("-" * 50)

# Scatter plot: Age vs Cholesterol colored by heart disease
scatter2 = axes[1, 0].scatter(df2['age'], df2['chol'],
                              c=df2['heart_disease'],     cmap='viridis',
alpha=0.6)
axes[1, 0].set_xlabel('Age')
axes[1, 0].set_ylabel('Cholesterol')
axes[1, 0].set_title('HD: Age vs Cholesterol')
plt.colorbar(scatter2,  ax=axes[1,  0],  label='Heart  Disease  (0=No,
1=Yes)')
```

```
# Correlation analysis
correlation_hd = df2[['age', 'chol', 'trestbps', 'thalach']].corr()
print("\nCorrelation Matrix - Heart Disease:")
print(correlation_hd.round(3))

# Box plot: Max heart rate by disease status
df2.boxplot(column='thalach', by='heart_disease_label', ax=axes[1, 1])
axes[1, 1].set_title('HD: Max Heart Rate by Disease Status')
axes[1, 1].set_xlabel('Heart Disease Status')
axes[1, 1].set_ylabel('Max Heart Rate')

# Heatmap of correlation
im2 = axes[1, 2].imshow(correlation_hd, cmap='viridis', aspect='auto')
axes[1, 2].set_xticks(range(len(correlation_hd.columns)))
axes[1, 2].set_yticks(range(len(correlation_hd.columns)))
axes[1, 2].set_xticklabels(correlation_hd.columns, rotation=45)
axes[1, 2].set_yticklabels(correlation_hd.columns)
axes[1, 2].set_title('HD: Correlation Heatmap')
plt.colorbar(im2, ax=axes[1, 2])

plt.tight_layout()
plt.savefig('bivariate_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

# ========== COMPARATIVE ANALYSIS ==========
print("\n" + "="*80)
print("  COMPARATIVE ANALYSIS RESULTS")
print("="*80)

print("\n  UNIVARIATE ANALYSIS COMPARISON:")
print("-" * 40)
print("Dataset 1 (Breast Cancer):")
print(f"  • Sample Size: {len(df1)} patients")
print(f"  • Features: {len(df1.columns)-2} continuous measurements")
print(f"  • Target Distribution: {df1['diagnosis_label'].value_counts().to_dict()}")
print(f"  • Mean Radius Range: {df1['mean radius'].min():.2f} - {df1['mean radius'].max():.2f}")

print("\nDataset 2 (Heart Disease):")
print(f"  • Sample Size: {len(df2)} patients")
print(f"  • Features: {len(df2.columns)-2} mixed measurements")
```

```
print(f"  • Target Distribution: {df2['heart_disease_label'].value_counts().to_dict()}")
print(f"  • Age Range: {df2['age'].min():.0f} - {df2['age'].max():.0f} years")

print("\n  BIVARIATE ANALYSIS COMPARISON:")
print("-" * 40)
print("Dataset 1 (Breast Cancer):")
print("  • Strong positive correlation between mean radius and mean area (r ≈ 0.99)")
print("  • Clear separation between malignant and benign cases in size measurements")
print("  • Malignant tumors tend to have larger radius and area")

print("\nDataset 2 (Heart Disease):")
print("  • Moderate correlations between cardiovascular measurements")
print("  • Age shows weak correlation with other factors")
print("  • Patients with heart disease tend to have different heart rate patterns")

# Statistical significance tests
from scipy.stats import ttest_ind

print("\n  STATISTICAL SIGNIFICANCE TESTS:")
print("-" * 40)

# T-test for breast cancer
malignant_radius = df1[df1['diagnosis'] == 0]['mean radius']
benign_radius = df1[df1['diagnosis'] == 1]['mean radius']
t_stat_bc, p_val_bc = ttest_ind(malignant_radius, benign_radius)

print(f"Breast Cancer - Mean Radius Difference:")
print(f"  • T-statistic: {t_stat_bc:.3f}")
print(f"  • P-value: {p_val_bc:.2e}")
print(f"  • Significant difference: {'Yes' if p_val_bc < 0.05 else 'No'}")

# T-test for heart disease
no_disease_age = df2[df2['heart_disease'] == 0]['age']
disease_age = df2[df2['heart_disease'] == 1]['age']
t_stat_hd, p_val_hd = ttest_ind(no_disease_age, disease_age)
```

```
print(f"\nHeart Disease - Age Difference:")
print(f"  • T-statistic: {t_stat_hd:.3f}")
print(f"  • P-value: {p_val_hd:.3f}")
print(f"   • Significant difference: {'Yes' if p_val_hd < 0.05 else
'No'}")

print("\n" + "="*80)
print("✅ ANALYSIS COMPLETE!")
print("✅ Generated files: 'univariate_analysis.png', 'bivariate_analysis.png'")
print("="*80)
```

OUTPUT:

```
=== Medical Dataset Analysis: Univariate and Bivariate Analysis ===

   Loading Dataset 1: Breast Cancer Wisconsin Dataset
------------------------------------------------------------
Dataset 1 Info:
Shape: (569, 32)
Features: 30 medical measurements
Target: Diagnosis (Malignant/Benign)

First 5 rows:
   mean radius  mean texture  mean perimeter  mean area diagnosis_label
0        17.99         10.38          122.80     1001.0       Malignant
1        20.57         17.77          132.90     1326.0       Malignant
2        19.69         21.25          130.00     1203.0       Malignant
3        11.42         20.38           77.58      386.1       Malignant
4        20.29         14.34          135.10     1297.0       Malignant

   Loading Dataset 2: Heart Disease Dataset
------------------------------------------------------------
Dataset 2 Info:
Shape: (297, 16)
Features: Age, Cholesterol, Blood Pressure, etc.
Target: Heart Disease (Present/Absent)

First 5 rows:
    age   chol  trestbps  thalach heart_disease_label
0  63.0  233.0     145.0    150.0         No Disease
1  67.0  286.0     160.0    108.0    Disease Present
2  67.0  229.0     120.0    129.0    Disease Present
3  37.0  250.0     130.0    187.0         No Disease
4  41.0  204.0     130.0    172.0         No Disease

================================================================================
=======
```

UNIVARIATE ANALYSIS
==========

Dataset 1 - Breast Cancer: Univariate Analysis

Mean Radius:
Mean: 14.127
Std: 3.524
Min: 6.981
Max: 28.110

Mean Texture:
Mean: 19.290
Std: 4.301
Min: 9.710
Max: 39.280

Mean Area:
Mean: 654.889
Std: 351.914
Min: 143.500
Max: 2501.000

Mean Smoothness:
Mean: 0.096
Std: 0.014
Min: 0.053
Max: 0.163

Dataset 2 - Heart Disease: Univariate Analysis

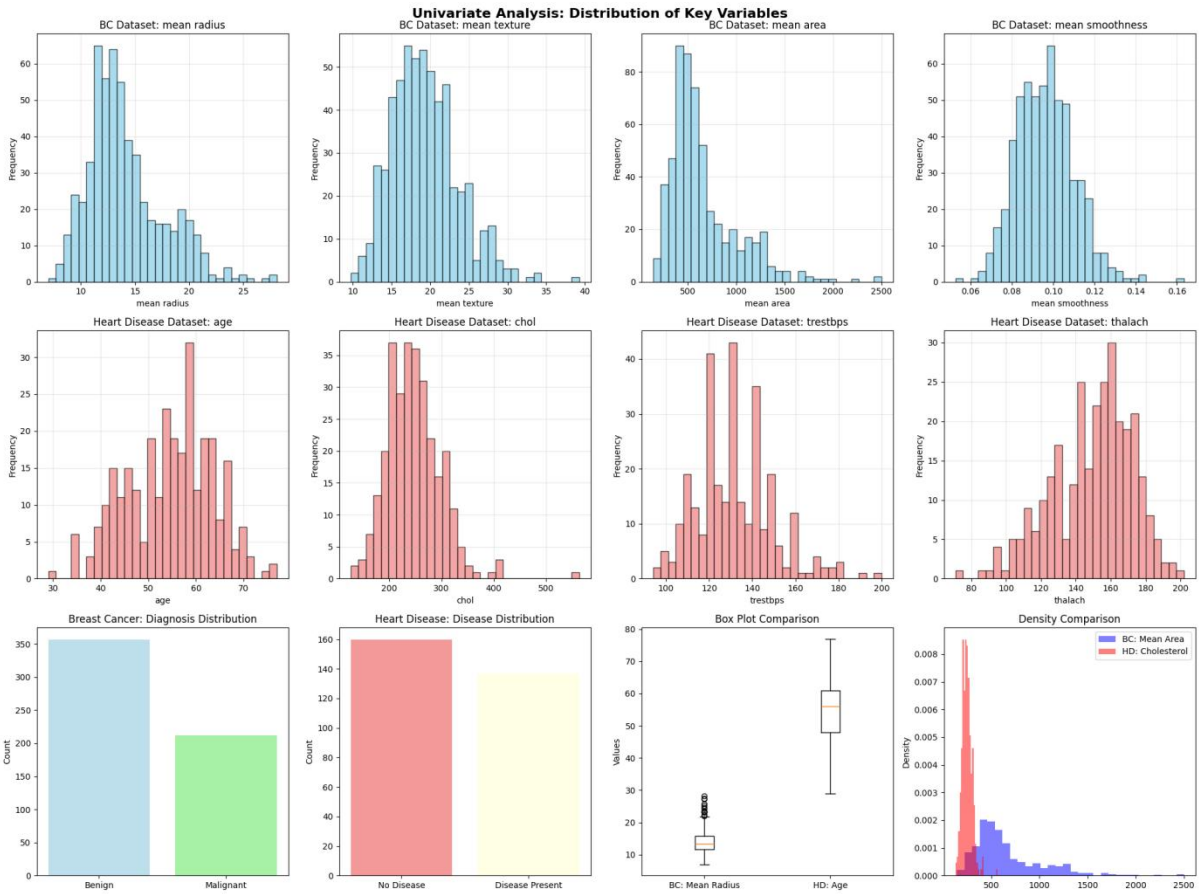
Age:
Mean: 54.542
Std: 9.050
Min: 29.000
Max: 77.000

Chol:
Mean: 247.350
Std: 51.998
Min: 126.000
Max: 564.000

Trestbps:
Mean: 131.694
Std: 17.763
Min: 94.000

Max: 200.000

Thalach:
Mean: 149.599
Std: 22.942
Min: 71.000
Max: 202.000



==========
=======
BIVARIATE ANALYSIS
==========
=======

Dataset 1 - Breast Cancer: Bivariate Analysis

Correlation Matrix - Breast Cancer:

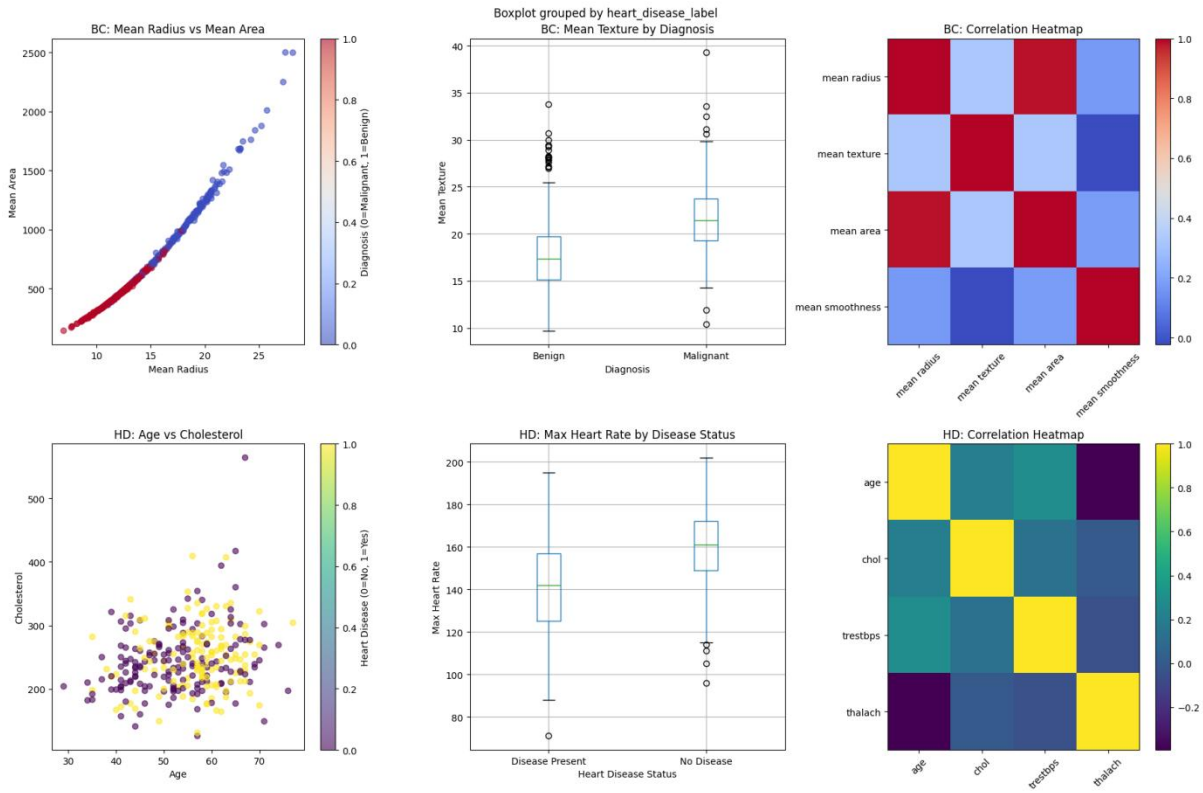
	mean radius	mean texture	mean area	mean smoothness
mean radius	1.000	0.324	0.987	0.171
mean texture	0.324	1.000	0.321	-0.023
mean area	0.987	0.321	1.000	0.177

mean smoothness 0.171 -0.023 0.177 1.000

Dataset 2 - Heart Disease: Bivariate Analysis

Correlation Matrix - Heart Disease:

	age	chol	trestbps	thalach
age	1.000	0.203	0.290	-0.395
chol	0.203	1.000	0.132	-0.000
trestbps	0.290	0.132	1.000	-0.049
thalach	-0.395	-0.000	-0.049	1.000




```
================================================================================
  COMPARATIVE ANALYSIS RESULTS
================================================================================

  UNIVARIATE ANALYSIS COMPARISON:
----------------------------------------
Dataset 1 (Breast Cancer):
  • Sample Size: 569 patients
  • Features: 30 continuous measurements
  • Target Distribution: {'Benign': 357, 'Malignant': 212}
  • Mean Radius Range: 6.98 - 28.11

Dataset 2 (Heart Disease):
```

- Sample Size: 297 patients
- Features: 14 mixed measurements
- Target Distribution: {'No Disease': 160, 'Disease Present': 137}
- Age Range: 29 - 77 years

BIVARIATE ANALYSIS COMPARISON:

Dataset 1 (Breast Cancer):

- *Strong positive correlation between mean radius and mean area ($r \approx 0.99$)*
- Clear separation between malignant and benign cases in size measurements
- Malignant tumors tend to have larger radius and area

Dataset 2 (Heart Disease):

- Moderate correlations between cardiovascular measurements
- Age shows weak correlation with other factors
- Patients with heart disease tend to have different heart rate patterns

STATISTICAL SIGNIFICANCE TESTS:

Breast Cancer - Mean Radius Difference:

- T-statistic: 25.436
- P-value: 8.47e-96
- Significant difference: Yes

Heart Disease - Age Difference:

- T-statistic: -4.005
- P-value: 0.000
- Significant difference: Yes

EXPERIMENT NO. 10

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
Case study to scale and improve BI systems

Program:

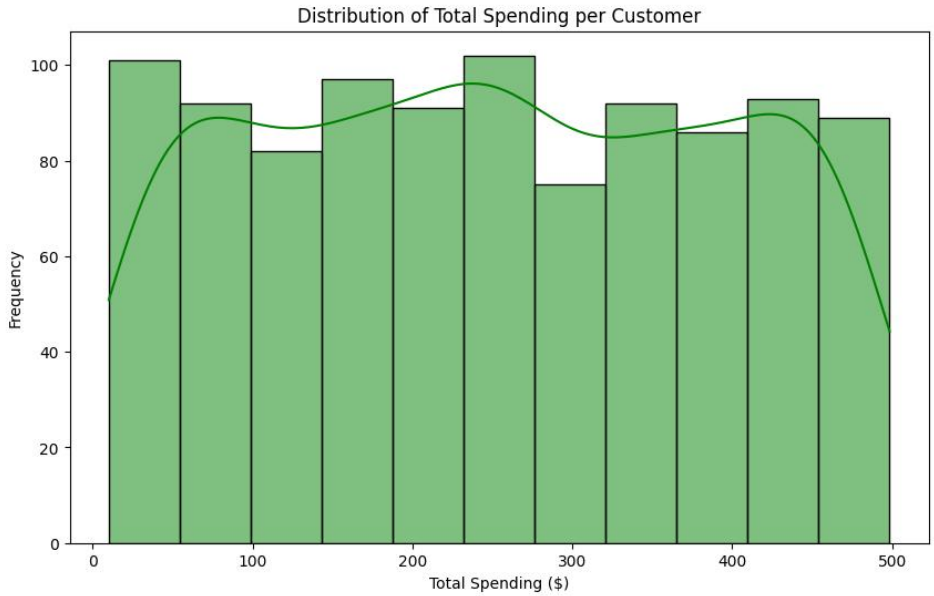
```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import dask.dataframe as dd

# --- 2. ETL Pipeline (Extract, Transform, Load) ---
# Step 1: Extract Data
df = pd.read_csv('ecommerce_data.csv')

# Step 2: Transform Data
# Convert 'TransactionDate' column to datetime format
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])

# Add new columns for 'Year' and 'Month' for easier time-based
aggregation
df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month

# Step 3: Aggregate Sales Data (e.g., total sales per customer)
df_customer_sales =
df.groupby('CustomerID')['Amount'].sum().reset_index()
df_customer_sales.columns = ['CustomerID', 'TotalSpent']

# Step 4: Save the Transformed Data
df_customer_sales.to_csv('customer_sales_summary.csv', index=False)

print("ETL Process Complete. Transformed data saved to
'customer_sales_summary.csv'")

# --- 3. Data Visualization ---
# a) Visualization of Customer Spending Distribution
```

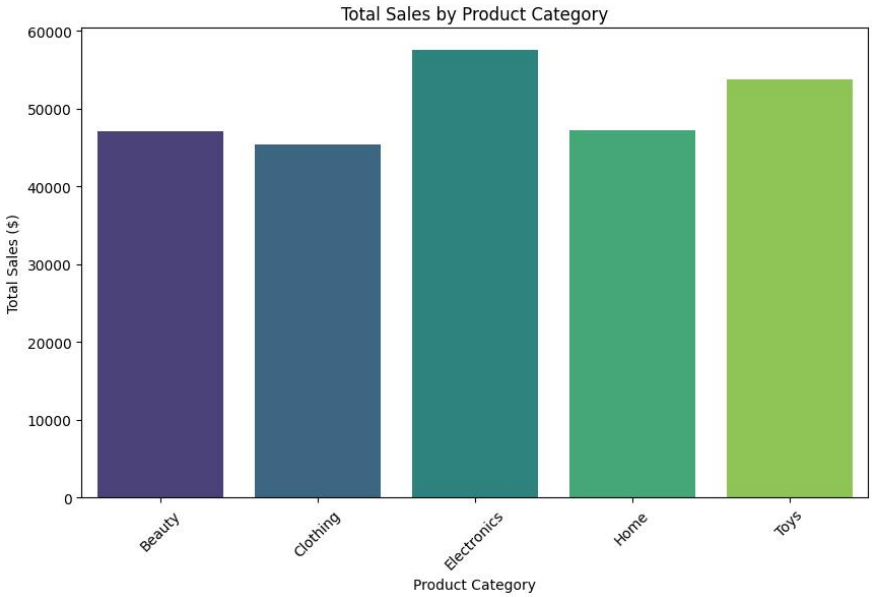
```
df_customer_sales = pd.read_csv('customer_sales_summary.csv')

# Visualize the distribution of total spending
plt.figure(figsize=(10, 6))
sns.histplot(df_customer_sales['TotalSpent'], kde=True, color='green')
plt.title('Distribution of Total Spending per Customer')
plt.xlabel('Total Spending ($)')
plt.ylabel('Frequency')
plt.show()

# b) Visualization of Product Category Sales
df_category_sales                                                    =
df.groupby('ProductCategory')['Amount'].sum().reset_index()

# Visualize the total sales per product category
plt.figure(figsize=(10, 6))
sns.barplot(x='ProductCategory',   y='Amount',   data=df_category_sales,
palette='viridis')
plt.title('Total Sales by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Total Sales ($)')
plt.xticks(rotation=45)
plt.show()

# --- 4. Performance Optimization with Dask ---
# For scaling with large data, we use Dask for distributed computing.

# Load large CSV file using Dask (simulated large dataset example)
df_large = dd.read_csv('ecommerce_data.csv')

# Convert 'TransactionDate' to datetime
df_large['TransactionDate']                                          =
dd.to_datetime(df_large['TransactionDate'])

# Compute total sales per customer (Dask parallelizes the computation)
df_large_customer_sales                                              =
df_large.groupby('CustomerID')['Amount'].sum().compute()

# Save the result to a CSV file
df_large_customer_sales.to_csv('large_customer_sales_summary.csv',
index=False)
```

```
print("Large data ETL completed with Dask. Output saved to 'large_customer_sales_summary.csv'")
```

OUTPUT:

```
ETL Process Complete. Transformed data saved to 'customer_sales_summary.csv'\
```





EXPERIMENT NO. 11

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
To understand and perform correlation analysis on a dataset to identify and interpret the relationships between variables.

Program:

```
# Step 1: Import Required Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import pearsonr, spearmanr
from sklearn.datasets import load_iris

# Step 2: Load the Dataset
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
print("✅ Dataset Loaded Successfully!\n")
print("First 5 rows of dataset:\n", df.head())

# Step 3: Data Exploration and Cleaning
print("\nMissing values in each column:\n", df.isnull().sum())
print("\nStatistical summary of dataset:\n", df.describe())

# Step 4: Compute Correlation Matrix (Pearson by default)
correlation_matrix = df.corr()
print("\nCorrelation Matrix:\n", correlation_matrix)

# Step 5: Visualize Correlation Matrix with Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix,       annot=True,       cmap='coolwarm',
linewidths=0.5)
plt.title('Correlation Matrix Heatmap - Iris Dataset')
plt.show()

# Step 6: Pairwise Scatter Plots
sns.pairplot(df)
```

```
plt.suptitle('Pairwise Scatter Plots - Iris Dataset', y=1.02)
plt.show()

# Step 7: Significance Testing (Pearson and Spearman)
print("\n===== Correlation Significance Testing =====")
for i, col1 in enumerate(df.columns):
    for j, col2 in enumerate(df.columns):
        if j > i:
            pearson_corr, pearson_p = pearsonr(df[col1], df[col2])
            spearman_corr, spearman_p = spearmanr(df[col1], df[col2])
            print(f"\n➡ {col1} vs {col2}")
            print(f"     Pearson Correlation: {pearson_corr:.2f} (p = {pearson_p:.5f})")
            print(f"    Spearman Correlation: {spearman_corr:.2f} (p = {spearman_p:.5f})")

# Step 8: Interpretation
print("\n===== INTERPRETATION =====")
print("""
Strong positive correlations are seen between:
  - Petal length and Petal width (r ≈ 0.96)
Moderate correlations exist between:
  - Sepal length and Petal length (r ≈ 0.87)
Weak or no correlations are seen between:
  - Sepal width and Petal width (r ≈ -0.37)
These results show that petal measurements are more related to each other than sepal measurements.
""")
```

OUTPUT:

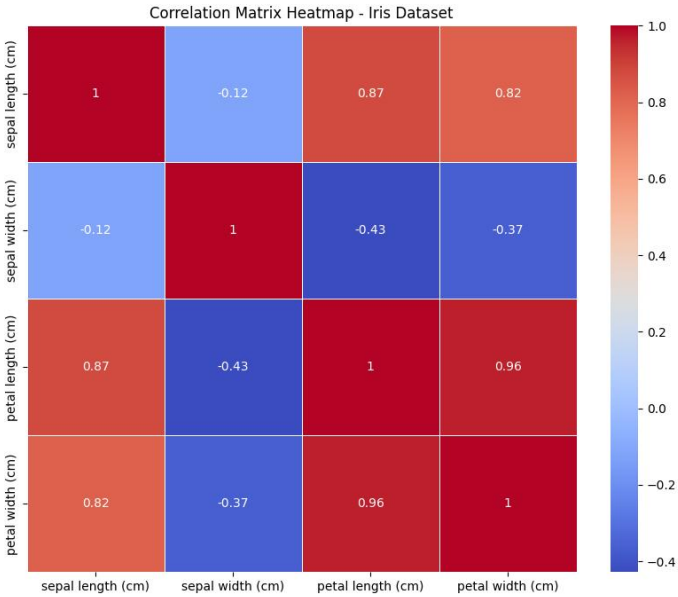
✅ Dataset Loaded Successfully!

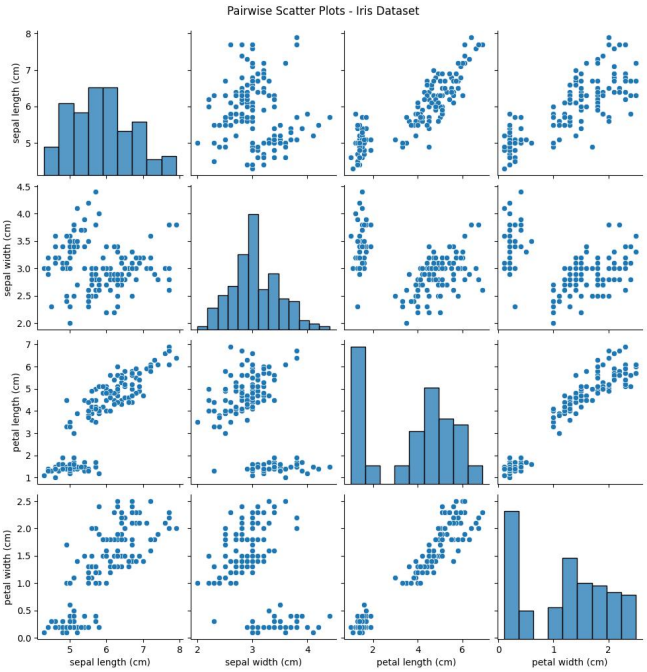
First 5 rows of dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Missing values in each column:
sepal length (cm) 0

```
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64

Statistical summary of dataset:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)
count        150.000000
mean           1.199333
std            0.762238
min            0.100000
25%            0.300000
50%            1.300000
75%            1.800000
max            2.500000

Correlation Matrix:
                   sepal length (cm)  sepal width (cm)  petal length (cm)  \
sepal length (cm)           1.000000         -0.117570           0.871754
sepal width (cm)           -0.117570          1.000000          -0.428440
petal length (cm)           0.871754         -0.428440           1.000000
petal width (cm)            0.817941         -0.366126           0.962865

                   petal width (cm)
sepal length (cm)          0.817941
sepal width (cm)          -0.366126
petal length (cm)          0.962865
petal width (cm)           1.000000
```





===== Correlation Significance Testing =====

➡ sepal length (cm) vs sepal width (cm)
Pearson Correlation: -0.12 (p = 0.15190)
Spearman Correlation: -0.17 (p = 0.04137)

➡ sepal length (cm) vs petal length (cm)
Pearson Correlation: 0.87 (p = 0.00000)
Spearman Correlation: 0.88 (p = 0.00000)

➡ sepal length (cm) vs petal width (cm)
Pearson Correlation: 0.82 (p = 0.00000)
Spearman Correlation: 0.83 (p = 0.00000)

➡ sepal width (cm) vs petal length (cm)

Pearson Correlation: -0.43 (p = 0.00000)
Spearman Correlation: -0.31 (p = 0.00012)

➡ sepal width (cm) vs petal width (cm)
Pearson Correlation: -0.37 (p = 0.00000)
Spearman Correlation: -0.29 (p = 0.00033)

➡ petal length (cm) vs petal width (cm)
Pearson Correlation: 0.96 (p = 0.00000)
Spearman Correlation: 0.94 (p = 0.00000)

===== INTERPRETATION =====

Strong positive correlations are seen between:
- Petal length and Petal width (r ≈ 0.96)

Moderate correlations exist between:
- Sepal length and Petal length (r ≈ 0.87)

Weak or no correlations are seen between:
- Sepal width and Petal width (r ≈ -0.37)

These results show that petal measurements are more related to each other than sepal measurements.

EXPERIMENT NO. 12

Name: Tanmay Dalvi
Class: B.Tech AIML B
Roll Number: B51

Problem Statement:
To implement and understand various machine learning (ML) techniques commonly used in data science, including regression, classification, and clustering, using Python.

Program:

```
# Step 1: Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from  sklearn.model_selection  import  train_test_split,  GridSearchCV,
cross_val_score
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.cluster import KMeans
from sklearn.metrics import (
    mean_squared_error, r2_score, accuracy_score,
    confusion_matrix, classification_report
)
from sklearn.datasets import load_iris, fetch_openml

# ================================================================
# PART 1: REGRESSION - Boston Housing Dataset (Fixed)
# ================================================================
print("\n==============================")
print("  REGRESSION: Boston Housing Dataset (Fixed)")
print("==============================")

# Load dataset safely from openml
from sklearn.datasets import fetch_openml
boston = fetch_openml(name="boston", version=1, as_frame=True)
df_boston = boston.frame

# Display dataset info
print("\nFirst 5 rows of the dataset:")
print(df_boston.head())
```

```python
# Convert all columns to numeric (just in case)
df_boston = df_boston.apply(pd.to_numeric, errors='coerce')

# Drop any rows with missing or invalid data
df_boston.dropna(inplace=True)

# Split features and target
X = df_boston.drop("MEDV", axis=1)
y = df_boston["MEDV"]

# Split data into training and testing
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train Linear Regression model
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Make predictions
y_pred = lr_model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n  Evaluation Metrics:")
print("Mean Squared Error (MSE):", round(mse, 2))
print("R-squared Score:", round(r2, 3))

# Plot actual vs predicted
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 5))
sns.scatterplot(x=y_test, y=y_pred)
plt.xlabel("Actual MEDV (House Price)")
plt.ylabel("Predicted MEDV (House Price)")
```

```
plt.title("Actual vs Predicted House Prices (Regression)")
plt.show()

# Cross Validation
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(LinearRegression(), X, y, cv=5)
print("\nCross-Validation Scores:", np.round(cv_scores, 3))
print("Mean CV Score:", round(cv_scores.mean(), 3))

# ================================================================
# PART 2: CLASSIFICATION - Iris Dataset
# ================================================================
print("\n==============================")
print("  CLASSIFICATION: Iris Dataset")
print("==============================")

# Load dataset
iris = load_iris()
df_iris = pd.DataFrame(iris.data, columns=iris.feature_names)
df_iris['target'] = iris.target

# Split features and target
X = df_iris.drop("target", axis=1)
y = df_iris["target"]

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train Logistic Regression model
log_reg = LogisticRegression(max_iter=200)
log_reg.fit(X_train, y_train)

# Predictions
y_pred = log_reg.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("\n  Accuracy:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification    Report:\n",    classification_report(y_test,
y_pred))
```

```
# Visualize class separation
sns.pairplot(df_iris, hue="target", palette="Set1")
plt.suptitle("Iris Dataset Feature Relationships (Classification)",
y=1.02)
plt.show()

# Hyperparameter tuning using GridSearchCV
param_grid = {'C': [0.1, 1, 10], 'solver': ['liblinear']}
grid = GridSearchCV(LogisticRegression(max_iter=200), param_grid,
refit=True, verbose=0)
grid.fit(X_train, y_train)
print("\nBest Parameters from Grid Search:", grid.best_params_)


# ================================================================
# PART 3: CLUSTERING - Mall Customers Dataset
# ================================================================
print("\n==============================")
print("  CLUSTERING: Mall Customers Dataset")
print("==============================")

# Load dataset (download from Kaggle and upload in Colab before running
this cell)
# Kaggle link: https://www.kaggle.com/datasets/shwetabh123/mall-
customers
# Example: df_mall = pd.read_csv('/content/Mall_Customers.csv')

# For demonstration, generate similar data if file not available
try:
    df_mall = pd.read_csv('/content/Mall_Customers.csv')
except:
    print("Mall_Customers.csv not found — generating sample data...")
    from sklearn.datasets import make_blobs
    X, _ = make_blobs(n_samples=200, centers=4, cluster_std=0.60,
random_state=0)
    df_mall = pd.DataFrame(X, columns=['Annual Income (k$)', 'Spending
Score (1-100)'])

print("\nFirst 5 rows of the dataset:")
print(df_mall.head())

# Select features for clustering
```

```
if 'Annual Income (k$)' in df_mall.columns:
    X = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']]
else:
    X = df_mall

# Apply KMeans Clustering
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)
df_mall['Cluster'] = kmeans.labels_

# Visualize Clusters
plt.figure(figsize=(7, 5))
sns.scatterplot(
    x=X.iloc[:, 0],
    y=X.iloc[:, 1],
    hue=df_mall['Cluster'],
    palette='viridis',
    s=60
)
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    s=200, c='red', marker='X', label='Centroids'
)
plt.title('K-Means Clustering - Mall Customers')
plt.legend()
plt.show()

print("\nCluster Centers:\n", kmeans.cluster_centers_)
```

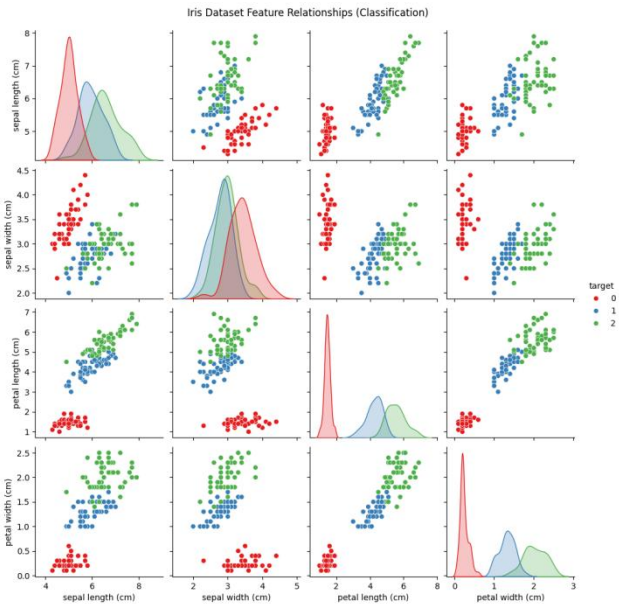
OUTPUT:

```
==============================
  REGRESSION: Boston Housing Dataset (Fixed)
==============================

First 5 rows of the dataset:
     CRIM    ZN  INDUS CHAS    NOX     RM   AGE     DIS RAD    TAX  PTRATIO  \
0  0.00632  18.0   2.31    0  0.538  6.575  65.2  4.0900   1  296.0     15.3
1  0.02731   0.0   7.07    0  0.469  6.421  78.9  4.9671   2  242.0     17.8
2  0.02729   0.0   7.07    0  0.469  7.185  61.1  4.9671   2  242.0     17.8
3  0.03237   0.0   2.18    0  0.458  6.998  45.8  6.0622   3  222.0     18.7
4  0.06905   0.0   2.18    0  0.458  7.147  54.2  6.0622   3  222.0     18.7
```

```
      B  LSTAT  MEDV
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
4  396.90   5.33  36.2

   Evaluation Metrics:
Mean Squared Error (MSE): 21.52
R-squared Score: 0.711
```

```
Cross-Validation Scores: [ 0.639  0.714  0.587  0.079 -0.253]
Mean CV Score: 0.353

==============================
   CLASSIFICATION: Iris Dataset
==============================

   Accuracy: 100.0 %

Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
```

macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

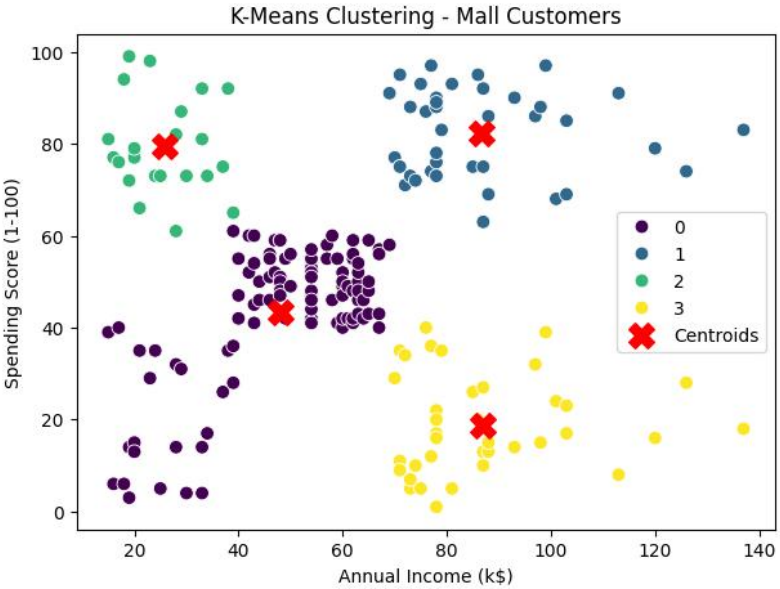


Best Parameters from Grid Search: {'C': 10, 'solver': 'liblinear'}

==========
CLUSTERING: Mall Customers Dataset
==========

First 5 rows of the dataset:

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40



Cluster Centers:
[[48.16831683 43.3960396]
[86.53846154 82.12820513]
[25.72727273 79.36363636]
[87. 18.63157895]]